

疯狂打工人有限公司

Crazy Worker (英文名) • V1.3.0

Source code excerpt (first 30 + last 30 pages)

Scope: Assets_Game own source. Files: 33 / Lines: 10874 / 50 lines per page / Total pages: 218

Generated: 2026-09-21 15:52:55 (local build chain, headless Edge print)

```
// ===== FILE: Assets\_Game\Scripts\Editor\ArtWireCheck.cs =====
using System;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;
using Sy.G11.Game;

namespace Sy.G11
{
    /// <summary>
    /// T016 P-slice machine gate: art stem registry + placeholder wiring (X027
    /// two-state law) ahead of the D-tier art production window.
    /// W1 registry_law : AllT016Stems returns exactly the fifteen S1 work-order
    /// stems of art requirement doc section 7, all distinct.
    /// W2 fallback_law : every stem resolves through WorkerPlaceholder.Face with
    /// no catalog art (null-safe procedural fallback alive),
    /// double-call returns the cached instance and the raw
    /// procedural texture is byte-identical across rebuilds.
    /// W3 casting_law : every frozen event id (incl. chain rows) casts to one of
    /// the four character stems, all four are used, mapping is
    /// deterministic.
    /// W4 hud_icon_law : the three wired HUD icon stems are distinct registry
    /// members (A10/A11/A12).
    /// W5 slice_law : the nine sliced sub-key cells (badge_tiers_0..4 +
    /// skills_0..3, the X408 swap points) resolve to real
    /// catalog sprites after the A13/A14 sheet slicing.
    /// The office rank binding law stays owned by OfflineCheck 06 (frozen); the
    /// six-face determinism sweep stays owned by DataSchemaCheck (frozen).
    /// Usage: Tuanjie.exe -batchmode -quit -executeMethod Sy.G11.ArtWireCheck.Run -logFile <abs>
    /// Exit code: 0 = all pass, 1 = assertion failure, 2 = exception.
    /// </summary>
    public static class ArtWireCheck
    {
        // Frozen doc list (art requirement doc section 7, A01-A15 order).
        static readonly string[] DocStems =
        {
            "G11_unit_01_intern",
            "G11_unit_02_boss_wang",
            "G11_unit_03_hr_tian",
            "G11_unit_04_kevin",
            "G11_unit_05_master_zhang",
            "G11_scene_office_open",
            "G11_scene_office_cubicle",
            "G11_scene_office_private",
            "G11_scene_office_penthouse",
            "G11_icon_ui_coin",
            "G11_icon_ui_mood",
            "G11_icon_ui_kpi",
            "G11_icon_badge_tiers",

```

```

        "G11_icon_skills",
        "G11_event_card_frame"
    };

    public static void Run()
    {
        int code;
        try
        {
            bool w1 = RegistryLaw();
            bool w2 = FallbackLaw();
            bool w3 = CastingLaw();
            bool w4 = HudIconLaw();
            bool w5 = SliceLaw();
            bool all = w1 && w2 && w3 && w4 && w5;

            var sb = new StringBuilder();
            sb.AppendLine("G11 T016 art wire machine gate, P-slice (X027 two-state law, 15 stems)");
            sb.AppendLine($"W1 registry_law => {(w1 ? "PASS" : "FAIL")} (AllT016Stems == doc section 7 list, 15 distinct)");
            sb.AppendLine($"W2 fallback_law => {(w2 ? "PASS" : "FAIL")} (null-safe Face resolve + cache + byte-identical
procedural rebuild)");
            sb.AppendLine($"W3 casting_law => {(w3 ? "PASS" : "FAIL")} (every event id casts to a unit stem, 4/4 used,
deterministic)");
            sb.AppendLine($"W4 hud_icon_law => {(w4 ? "PASS" : "FAIL")} (A10/A11/A12 distinct registry members)");
            sb.AppendLine($"W5 slice_law => {(w5 ? "PASS" : "FAIL")} (9 sliced sub-key sprites registered: badge_tiers_0..4 +
skills_0..3)");
            sb.AppendLine($"RESULT => {(all ? "ALL PASS" : "FAIL")}");

            var logsDir = Path.Combine(Path.GetFullPath(Path.Combine(Application.dataPath, "..")), "Logs");
            if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
            File.WriteAllText(Path.Combine(logsDir, "g11_artwire_report.txt"), sb.ToString(), new UTF8Encoding(false));

            foreach (var line in sb.ToString().Split('\n')) Debug.Log("[ArtWireCheck] " + line.TrimEnd());
            code = all ? 0 : 1;
        }
        catch (System.Exception e)
        {
            Debug.LogError("[ArtWireCheck] exception: " + e);
            code = 2;
        }
        EditorApplication.Exit(code);
    }

    // ----- W1: registry equals the frozen doc work orders -----

    static bool RegistryLaw()
    {
        var stems = WorkerPlaceholder.AllT016Stems();
        if (stems == null || stems.Length != 15) return false;
        for (int i = 0; i < stems.Length; i++)
        {
            if (string.IsNullOrEmpty(stems[i])) return false;
            for (int j = i + 1; j < stems.Length; j++)

```

```

        if (stems[i] == stems[j]) return false;
    }
    for (int i = 0; i < DocStems.Length; i++)
    {
        bool found = false;
        for (int j = 0; j < stems.Length; j++)
            if (stems[j] == DocStems[i]) { found = true; break; }
        if (!found) return false;
    }
    return true;
}

// ----- W2: two-state fallback alive for all 15 stems -----

static bool FallbackLaw()
{
    var stems = WorkerPlaceholder.AllT016Stems();
    for (int i = 0; i < stems.Length; i++)
    {
        // catalog art is absent in this slice, so Face must resolve the
        // procedural fallback - never null (the null-safe wiring contract)
        var s1 = WorkerPlaceholder.Face(stems[i], 64, WorkerPlaceholder.BaseColorFor(stems[i]));
        if (s1 == null) return false;
        var s2 = WorkerPlaceholder.Face(stems[i], 64, WorkerPlaceholder.BaseColorFor(stems[i]));
        if (!ReferenceEquals(s1, s2)) return false; // cache law, zero rebuild
        var t1 = WorkerPlaceholder.Procedural(stems[i], 64, WorkerPlaceholder.BaseColorFor(stems[i]));
        var t2 = WorkerPlaceholder.Procedural(stems[i], 64, WorkerPlaceholder.BaseColorFor(stems[i]));
        var p1 = t1.GetPixels32();
        var p2 = t2.GetPixels32();
        if (p1.Length != p2.Length) return false;
        for (int p = 0; p < p1.Length; p++)
            if (p1[p].r != p2[p].r || p1[p].g != p2[p].g || p1[p].b != p2[p].b) return false;
        UnityEngine.Object.DestroyImmediate(t1);
        UnityEngine.Object.DestroyImmediate(t2);
    }
    return true;
}

// ----- W3: event portrait casting covers the frozen table -----

static bool CastingLaw()
{
    var unitStems = new[]
    {
        WorkerPlaceholder.StemUnitBossWang, WorkerPlaceholder.StemUnitHrTian,
        WorkerPlaceholder.StemUnitKevin, WorkerPlaceholder.StemUnitMasterZhang
    };
    var used = new bool[4];
    var events = WorkerData.Events;
    for (int i = 0; i < events.Length; i++)

```

```

    {
        string id = events[i].Id;
        string a = WorkerMainHost.EventPortraitFor(id);
        string b = WorkerMainHost.EventPortraitFor(id);
        if (a != b) return false; // deterministic
        int slot = Array.IndexOf(unitStems, a);
        if (slot < 0) return false; // casts only to character stems
        used[slot] = true;
    }
    for (int i = 0; i < used.Length; i++)
        if (!used[i]) return false; // all four characters earn screen time
    return true;
}

// ----- W5: sliced sub-key cells carry real catalog art -----

static bool SliceLaw()
{
    for (int r = 0; r < 5; r++)
        if (Sy.Core.ArtRuntime.Get(WorkerPlaceholder.StemIconBadgeTiers + "_" + r) == null) return false;
    for (int i = 0; i < 4; i++)
        if (Sy.Core.ArtRuntime.Get(WorkerPlaceholder.StemIconSkills + "_" + i) == null) return false;
    return true;
}

// ----- W4: wired HUD icons are registry members -----

static bool HudIconLaw()
{
    var hud = WorkerMainHost.HudIconStems();
    if (hud == null || hud.Length != 3) return false;
    var registry = WorkerPlaceholder.AllT016Stems();
    for (int i = 0; i < hud.Length; i++)
    {
        for (int j = i + 1; j < hud.Length; j++)
            if (hud[i] == hud[j]) return false; // distinct
        bool member = false;
        for (int r = 0; r < registry.Length; r++)
            if (registry[r] == hud[i]) { member = true; break; }
        if (!member) return false;
    }
    return true;
}
}

}

// ===== FILE: Assets\_Game\Scripts\Editor\BootSequenceCheck.cs =====
using System.IO;
using System.Text;
using UnityEditor;
using UnityEditor.SceneManagement;

```

```

using UnityEngine;

namespace Sy.G11
{
    /// <summary>
    /// Boot涓?寫?dry-run鑄空019鑄?徽鑄宜atchmode鑄?彗鑄?細
    /// Tuanjie.exe -batchmode -quit -executeMethod Sy.G11.BootSequenceCheck.Run -logFile <abs>
    /// 閩€錢虹燦鑄?=涓?寫?鑄?€?氫?饒煎紡淪岫嘖 1=澶?辨觸/寮?傲父 3=鑄烘樞鏈?啟錫情紙鑄塢窰ProjectSetup鑄?
    /// </summary>
    public static class BootSequenceCheck
    {
        public static void Run()
        {
            int code;
            try
            {
                string mainAsset = "Assets/_Game/Scenes/S11_Main.unity";
                string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
                if (!File.Exists(Path.Combine(projectRoot, mainAsset)))
                {
                    Debug.LogError("[BootSequenceCheck] main scene missing - run ProjectSetup first");
                    EditorApplication.Exit(3);
                    return;
                }

                Boot.BootServices.RegisterAll();
                bool switched = false;
                var sequence = new Boot.BootSequence(delegate
                {
                    EditorSceneManager.OpenScene(mainAsset, OpenSceneMode.Single);
                    switched = true;
                    return true;
                });
                bool allOk = sequence.RunAll();

                var sb = new StringBuilder();
                foreach (var line in sequence.Lines) sb.AppendLine(line);
                var logsDir = Path.Combine(projectRoot, "Logs");
                if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
                File.WriteAllText(Path.Combine(logsDir, "boot_sequence_report.txt"), sb.ToString(), new UTF8Encoding(false));

                bool wellFormed = sequence.Lines.Count == Boot.BootSequence.TotalSteps && switched;
                for (int i = 0; i < sequence.Lines.Count; i++)
                {
                    Debug.Log(sequence.Lines[i]);
                    if (!sequence.Lines[i].StartsWith("[BOOT] " + (i + 1) + "/" + Boot.BootSequence.TotalSteps + " "))
                        wellFormed = false;
                }

                code = allOk && wellFormed ? 0 : 1;
            }
            catch { }
        }
    }
}

```

```

    }
    catch (System.Exception e)
    {
        Debug.LogError("[BootSequenceCheck] exception: " + e);
        code = 1;
    }
    EditorApplication.Exit(code);
}
}

}
// ===== FILE: Assets\_Game\Scripts\Editor\BuildService.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;
using System.Text;
using UnityEditor;
using UnityEditor.Build;
using UnityEngine;

namespace Sy.G11
{
    /// <summary>
    /// X017宸 ュ 町 閩 句 竺 浞 踪 罇 湔 嬖 竺 铸 氮 纒 璇 戰 沃 璫 儻 嬖 鑛 ュ 々 铸 玟atchmode緯 ュ 彝 铸 条€?
    /// 鑾 儿 磅 铸 歌uanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.G11.BuildService.CompileCheck -logFile <緇 漬 罇 璽 罇 緞>
    /// 閩€鍍 虹 燼 铸?=閩 墮 枝 璇 罇 沃 璫 儻 1=瀛 樺 濠error 2=瀛 樺 濠warning铸 圖 oD 緇 罇 鑛 永 沃 璫 儻 緋 艳 瘡 铸? 3=鏈 罇 濁 渚?logFile 4=寮 傲 父
    /// </summary>
    public static class BuildService
    {
        public const int Ok = 0;
        public const int HasError = 1;
        public const int HasWarning = 2;
        public const int NoLogFile = 3;
        public const int Exception = 4;

        public static void CompileCheck()
        {
            var report = new CompileCheckReport
            {
                project = "G11_CrazyWorker",
                unityVersion = Application.unityVersion,
                atUtc = DateTime.UtcNow.ToString("o"),
                errors = new List<string>(),
                warnings = new List<string>(),
            };

            int code = Ok;
            try
            {
                string logPath = FindArgValue("-logFile");
            }
        }
    }
}

```

```

        if (string.IsNullOrEmpty(logPath) || !File.Exists(logPath))
        {
            report.status = "no_log_file";
            code = NoLogFile;
        }
        else
        {
            ScanLog(logPath, report);
            if (report.errors.Count > 0) { report.status = "error"; code = HasError; }
            else if (report.warnings.Count > 0) { report.status = "warning"; code = HasWarning; }
            else { report.status = "ok"; code = 0k; }
        }
    }
    catch (Exception e)
    {
        report.status = "exception";
        report.exception = e.Message + "\n" + e.StackTrace;
        code = Exception;
    }

    WriteReport(report);
    EditorApplication.Exit(code);
}

// <summary>涕床悽涕よ回婆俗贈漆槐€硃紙鏽回寔 -logFile x 涓? -logFile=x 涓よ回囊(-)紡铄条€?/summary>
internal static string FindArgValue(string name)
{
    var args = Environment.GetCommandLineArgs();
    for (int i = 0; i < args.Length; i++)
    {
        if (string.Equals(args[i], name, StringComparison.OrdinalIgnoreCase))
        {
            if (i + 1 < args.Length) return args[i + 1];
        }
        else if (args[i].StartsWith(name + "=", StringComparison.OrdinalIgnoreCase))
        {
            return args[i].Substring(name.Length + 1);
        }
    }
    return null;
}

// <summary>緇熒回緋栳栳error/warning铄玫atchmode涓懦紕璣賤所浜霸xecuteMethod铄岫回鏃舵欄塞樞凡纖界涓涓铄条€?
// Multi-pass guard: a batch session can emit a transient first-pass error set
// (new .cs files are imported and recompiled inside the same session, so the
// first pass may compile against a file set that does not contain them yet),
// then complete a clean final pass marked by the last "CompileScripts:" line.
// Only diagnostics emitted after the LAST completed pass count - evidence:
// 2026-09-20 first T006-a run, first-pass CS0246 x5 all cured by the in-session
// import + recompile, log order proves the final pass is green.</summary>

```

```

static void ScanLog(string logPath, CompileCheckReport report)
{
    var lines = new List<string>();
    using (var fs = new FileStream(logPath, FileMode.Open, FileAccess.Read, FileShare.ReadWrite))
    using (var sr = new StreamReader(fs, Encoding.UTF8))
    {
        string line;
        while ((line = sr.ReadLine()) != null) lines.Add(line);
    }

    int scanFrom = 0;
    for (int i = 0; i < lines.Count; i++)
    {
        if (lines[i].IndexOf("CompileScripts:", StringComparison.Ordinal) >= 0) scanFrom = i + 1;
    }

    var seenErrors = new HashSet<string>();
    var seenWarnings = new HashSet<string>();
    for (int i = scanFrom; i < lines.Count; i++)
    {
        var line = lines[i];
        if (line.IndexOf(": error CS", StringComparison.Ordinal) >= 0
            || line.IndexOf("Scripts have compiler errors", StringComparison.Ordinal) >= 0
            || line.IndexOf("Error building Player", StringComparison.Ordinal) >= 0)
        {
            if (seenErrors.Add(Trim(line))) report.errors.Add(Trim(line));
        }
        else if (line.IndexOf(": warning CS", StringComparison.Ordinal) >= 0)
        {
            if (seenWarnings.Add(Trim(line))) report.warnings.Add(Trim(line));
        }
    }
    report.errorCount = report.errors.Count;
    report.warningCount = report.warnings.Count;
}

static string Trim(string s)
{
    s = s.TrimEnd();
    return s.Length > 500 ? s.Substring(0, 500) + "... " : s;
}

static void WriteReport(CompileCheckReport report)
{
    try
    {
        var projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
        var logsDir = Path.Combine(projectRoot, "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(

```

```

        Path.Combine(logsDir, "compile_report.json"),
        JsonUtility.ToJson(report, true),
        new UTF8Encoding(false));
    Console.WriteLine($"[CompileCheck] status={report.status} errors={report.errorCount} warnings={report.warningCount} -
> Logs/compile_report.json");
    }
    catch (Exception e)
    {
        Console.WriteLine("[CompileCheck] report write failed: " + e.Message);
    }
}

/// <summary>
/// T011 slice 2: weixin milestone-package build entry (batchmode).
/// Ported from Sy.P06.BuildService.BuildWeixin (P02/P06 proven lineage):
/// reads pinned EditorBuildSettings scenes, switches to the WeixinMiniGame
/// target, pins the raw-container compression format (X123: brotli output
/// would break the later 4MB main-package split contract), builds with the
/// SY_WECHAT define pinned, and writes the machine-readable
/// Logs/build_weixin_report.json (result/totalSizeBytes/errors/warnings/
/// defineInjected/atUtc). The fmt pin is restored in finally.
/// Exit 0 = build succeeded, 1 = failure, 4 = exception.
/// Usage: Tuanjie.exe -batchmode -quit -projectPath &lt;p&gt; -executeMethod Sy.G11.BuildService.BuildWeixin -logFile
&lt;abs&gt;
/// </summary>
public static void BuildWeixin()
{
    int code;
    SerializedObject psObj = null;
    int fmtPrev = 0;
    bool pinned = false;
    try
    {
        var scenes = EditorBuildSettings.scenes;
        if (scenes == null || scenes.Length == 0)
        {
            Console.WriteLine("[BuildWeixin] no scenes pinned in EditorBuildSettings");
            EditorApplication.Exit(1);
            return;
        }
        var levellist = new List<string>();
        foreach (var s in scenes)
            if (s.enabled) levellist.Add(s.path);
        if (levellist.Count == 0)
        {
            Console.WriteLine("[BuildWeixin] no enabled scenes");
            EditorApplication.Exit(1);
            return;
        }

        bool switched = EditorUserBuildSettings.SwitchActiveBuildTarget(
            BuildTargetGroup.WeixinMiniGame, BuildTarget.WeixinMiniGame);

```

```

Console.WriteLine("[BuildWeixin] switchActiveTarget=" + switched
    + " scenes=" + levellist.Count);
if (!switched) { EditorApplication.Exit(1); return; }

string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
string outDir = Path.Combine(projectRoot, "MiniGame", "weixin", "build");
Directory.CreateDirectory(outDir);

// X123: pin the raw-container format before building (brotli
// output would break the later 4MB main-package split contract).
var psAll = UnityEngine.Resources.FindObjectsOfTypeAll(typeof(PlayerSettings));
if (psAll == null || psAll.Length == 0)
{
    Console.WriteLine("[BuildWeixin] X123 abort: PlayerSettings instance not found");
    EditorApplication.Exit(1);
    return;
}
psObj = new SerializedObject(psAll[0]);
var fmtProp = psObj.FindProperty("weixinMiniGameCompressionFormat");
if (fmtProp == null)
{
    Console.WriteLine("[BuildWeixin] X123 abort: serialized key weixinMiniGameCompressionFormat missing");
    EditorApplication.Exit(1);
    return;
}
fmtPrev = fmtProp.intValue;
fmtProp.intValue = 0;
psObj.ApplyModifiedProperties();
Console.WriteLine("[BuildWeixin] X123 fmtPin prev=" + fmtPrev
    + " set=0 readback=" + fmtProp.intValue);
if (fmtProp.intValue != 0)
{
    Console.WriteLine("[BuildWeixin] X123 abort: pin readback mismatch - no build");
    EditorApplication.Exit(1);
    return;
}
pinned = true;

string[] buildDefines = { "SY_WECHAT" };

var report = BuildPipeline.BuildPlayer(new BuildPlayerOptions
{
    scenes = levellist.ToArray(),
    locationPathName = outDir,
    target = BuildTarget.WeixinMiniGame,
    options = BuildOptions.None,
    extraScriptingDefines = buildDefines,
});

string result = report.summary.result.ToString();

```

```

long totalBytes = (long)report.summary.totalSize;
Console.WriteLine("[BuildWeixin] result=" + result
    + " totalSizeBytes=" + totalBytes
    + " errors=" + report.summary.totalErrors
    + " warnings=" + report.summary.totalWarnings);

try
{
    var logsDir = Path.Combine(projectRoot, "Logs");
    if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
    File.WriteAllText(
        Path.Combine(logsDir, "build_weixin_report.json"),
        "{ \"result\": \"" + result + "\", \"totalSizeBytes\": " + totalBytes
        + ", \"errors\": " + report.summary.totalErrors
        + ", \"warnings\": " + report.summary.totalWarnings
        + ", \"defineInjected\": \"" + string.Join(";", buildDefines) + "\" "
        + ", \"atUtc\": \"" + DateTime.UtcNow.ToString("o") + "\" }",
        new UTF8Encoding(false));
}
catch (Exception e) { Console.WriteLine("[BuildWeixin] report write failed: " + e.Message); }

code = report.summary.result == UnityEditor.Build.Reporting.BuildResult.Succeeded ? 0 : 1;
}
catch (Exception e)
{
    Console.WriteLine("[BuildWeixin] exception: " + e.Message + " | " + e.StackTrace);
    code = Exception;
}
finally
{
    if (pinned && psObj != null)
    {
        try
        {
            psObj.Update();
            var rp = psObj.FindProperty("weixinMiniGameCompressionFormat");
            if (rp != null)
            {
                rp.intValue = fmtPrev;
                psObj.ApplyModifiedProperties();
                Console.WriteLine("[BuildWeixin] X123 fmtRestored=" + rp.intValue);
            }
            else
            {
                Console.WriteLine("[BuildWeixin] X123 WARNING: restore key missing");
            }
        }
        catch (Exception rex)
        {
            Console.WriteLine("[BuildWeixin] X123 restore failed: " + rex.Message);
        }
    }
}

```

```

    }
}
}
EditorApplication.Exit(code);
}

// 鉅€鉅€ Douyin build face (MiniGame subplatform system, X170 lineage) 鉅€鉅€
// Ported from Sy.P06.BuildService.BuildDouyin (P02/P06 proven
// lineage, X121/X136 recipe): the douyin build rides the MiniGame
// platform (BuildTarget.MiniGame) with a subplatform selection, not
// a separate module; the stark editor assemblies do not sit in a
// fresh batchmode domain, so the sdk package dlls load explicitly
// via Assembly.LoadFrom and the carrier DouyinBuilder.BuildWebGL(
// StarkBuilderSettings) is reached through reflection. Every miss
// lands in Logs/build_douyin_report.json (honest failure surface).
// Only SY_DOUYIN is pinned transiently on the MiniGame named target
// (snapshot -> inject -> finally restore); the appId stays a
// placeholder until the console id lands at submission (user
// physical step).

internal static UnityEditor.MiniGameBuildSubtarget ResolveDouyinSubtarget()
{
    foreach (UnityEditor.MiniGameBuildSubtarget v in Enum.GetValues(typeof(UnityEditor.MiniGameBuildSubtarget)))
    {
        if (v.ToString().IndexOf("DouYin", StringComparison.OrdinalIgnoreCase) >= 0) return v;
    }
    throw new InvalidOperationException("MiniGameBuildSubtarget carries no DouYin member");
}

/// <summary>
/// Douyin build entry (batchmode). Creates the douyin build profile
/// (BuildTarget.MiniGame + douyin subtarget), auto-installs the
/// subplatform package when missing, then drives the X121-proven
/// carrier DouyinBuilder.BuildWebGL(StarkBuilderSettings) with the
/// fixed douyin profile while SY_DOUYIN is pinned on the MiniGame
/// named target (snapshot -> inject -> finally restore). Writes
/// Logs/build_douyin_report.json (status/subtarget/installed/verdict/
/// defineInjected/packageDir/atUtc). Exit 0 = package produced,
/// 1 = face miss or no package, 4 = exception.
/// Usage: Tuanjie.exe -batchmode -quit -executeMethod Sy.G11.BuildService.BuildDouyin -logFile &lt;abs&gt;
/// </summary>
public static void BuildDouyin()
{
    int code;
    string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
    string definesPrev = null;
    bool definePinned = false;
    var defineTarget = NamedBuildTarget.FromBuildTargetGroup(BuildTargetGroup.MiniGame);
    try
    {

```

```

var subtarget = ResolveDouyinSubtarget();
bool installed = UnityEditor.Build.Profile.BuildProfileSubplatformProxy.IsSubplatformInstalled(subtarget);
if (!installed)
    UnityEditor.Build.Profile.BuildProfileSubplatformProxy.InstallSubplatform(subtarget);
installed = UnityEditor.Build.Profile.BuildProfileSubplatformProxy.IsSubplatformInstalled(subtarget);

var profile = UnityEditor.Build.Profile.BuildProfile.CreateInstance(
    BuildTarget.MiniGame, subtarget);
if (profile == null)
{
    Console.WriteLine("[BuildDouyin] profile creation returned null");
    WriteDouyinReport(projectRoot, "no_profile", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}
string outDir = Path.Combine(projectRoot, "MiniGame", "douyin", "build");
Directory.CreateDirectory(outDir);
profile.buildPath = outDir;

// X136 law (P02 lineage): the helper.Build path is dead (X115:
// verdict=Unknown at 0.0s, the convert precondition never
// resolves there). The proven carrier is DouyinBuilder.
// BuildWebGL(StarkBuilderSettings) with the fixed douyin
// profile (placeholder appId, dev build, new build format,
// web-facing publish type, real output dir).
var settingsType = FindLoadedType("TTSDK.Tool.StarkBuilderSettings");
var douyinBuilderType = FindLoadedType("TTSDK.Tool.DouyinBuilder");
if (settingsType == null || douyinBuilderType == null)
{
    Console.WriteLine("[BuildDouyin] stark sdk types missing: settings="
        + (settingsType == null) + " builder=" + (douyinBuilderType == null));
    WriteDouyinReport(projectRoot, "no_stark_types", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}
MethodInfo loadMethod = null;
foreach (var m in settingsType.GetMethods(BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Static))
{
    var lp = m.GetParameters();
    if (m.Name == "LoadSettings" && lp.Length == 1 && lp[0].ParameterType == typeof(bool))
    { loadMethod = m; break; }
}
if (loadMethod == null)
{
    Console.WriteLine("[BuildDouyin] StarkBuilderSettings.LoadSettings(bool) not found");
    WriteDouyinReport(projectRoot, "no_settings_load", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}
object settings = loadMethod.Invoke(null, new object[] { true });

```

```

if (settings == null)
{
    Console.WriteLine("[BuildDouyin] LoadSettings(true) returned null");
    WriteDouyinReport(projectRoot, "settings_load_failed", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}
ApplyDouyinBuildProfile(settingsType, settings, outDir);

// The douyin helper path carries no extraScriptingDefines, so
// SY_DOUYIN is pinned on the MiniGame named target transiently
// (snapshot, mutate, restore in finally - the editor symbol
// table stays SY_* free between runs).
definesPrev = PlayerSettings.GetScriptingDefineSymbols(defineTarget);
var defineList = new List<string>();
foreach (var d in definesPrev.Split(';'))
{
    string dt = d.Trim();
    if (dt.Length > 0 && !defineList.Contains(dt)) defineList.Add(dt);
}
if (!defineList.Contains("SY_DOUYIN")) defineList.Add("SY_DOUYIN");
string definesMerged = string.Join(";", defineList);
PlayerSettings.SetScriptingDefineSymbols(defineTarget, definesMerged);
definePinned = true;
Console.WriteLine("[BuildDouyin] definePin prev='" + definesPrev + "' set='" + definesMerged + "'");

MethodInfo buildFace = null;
foreach (var m in douyinBuilderType.GetMethods(BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Static))
{
    var mp = m.GetParameters();
    if (m.Name == "BuildWebGL" && mp.Length == 1 && mp[0].ParameterType.Name == "StarkBuilderSettings")
    { buildFace = m; break; }
}
if (buildFace == null)
{
    Console.WriteLine("[BuildDouyin] DouyinBuilder.BuildWebGL(StarkBuilderSettings) not found");
    WriteDouyinReport(projectRoot, "no_build_face", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}

var watch = System.Diagnostics.Stopwatch.StartNew();
var buildTask = buildFace.Invoke(null, new object[] { settings }) as System.Threading.Tasks.Task;
while (buildTask != null && !buildTask.IsCompleted && watch.Elapsed.TotalSeconds < 900.0)
{
    System.Threading.Thread.Sleep(2000);
}
string verdict = buildTask == null ? "no_task"
    : (buildTask.IsCompleted ? buildTask.Status.ToString() : "pump_timeout");
if (buildTask != null && buildTask.Exception != null)

```

```

        verdict = buildTask.Status + "+" + buildTask.Exception.GetBaseException().GetType().Name;
        string packageDir = FindDouyinPackageDir(outDir);
        Console.WriteLine("[BuildDouyin] subtarget=" + subtarget
            + " installed=" + installed
            + " profileType=" + profile.GetType().Name
            + " taskSeconds=" + watch.Elapsed.TotalSeconds.ToString("F1")
            + " verdict=" + verdict
            + " packageDir=" + (packageDir ?? "none"));

        WriteDouyinReport(projectRoot,
            packageDir != null ? "succeeded" : "failed",
            subtarget.ToString(), installed, verdict, "SY_DOUYIN", packageDir);
        code = packageDir != null ? Ok : 1;
    }
    catch (Exception e)
    {
        Console.WriteLine("[BuildDouyin] exception: " + e.Message + " | " + e.StackTrace);
        WriteDouyinReport(projectRoot, "exception", string.Empty, false, e.Message);
        code = Exception;
    }
    finally
    {
        if (definePinned)
        {
            try
            {
                PlayerSettings.SetScriptingDefineSymbols(defineTarget, definesPrev);
                Console.WriteLine("[BuildDouyin] defineRestored="
                    + PlayerSettings.GetScriptingDefineSymbols(defineTarget) + "");
            }
            catch (Exception rex)
            {
                Console.WriteLine("[BuildDouyin] define restore failed: " + rex.Message);
            }
        }
    }
    EditorApplication.Exit(code);
}

static void WriteDouyinReport(string projectRoot, string status, string subtarget, bool installed, string verdict,
    string definesInjected = null, string packageDir = null)
{
    try
    {
        var logsDir = Path.Combine(projectRoot, "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(
            Path.Combine(logsDir, "build_douyin_report.json"),
            "{\n\"status\": \"" + status + "\",\n\"subtarget\": \"" + subtarget + "\"\n"
            + ",\n\"subplatformInstalled\": " + (installed ? "true" : "false")

```

```

        + "\",\"verdict\": \"" + (verdict ?? string.Empty).Replace("\\", "\\") + "\""
        + (string.IsNullOrEmpty(definesInjected) ? string.Empty : "\",\"defineInjected\": \"" + definesInjected + "\"")
        + (string.IsNullOrEmpty(packageDir) ? string.Empty : "\",\"packageDir\": \"" + packageDir.Replace("\\", "/") + "\"")
        + "\",\"atUtc\": \"" + DateTime.UtcNow.ToString("o") + "\"}",
        new UTF8Encoding(false));
    }
    catch (Exception e) { Console.WriteLine("[BuildDouyin] report write failed: " + e.Message); }
}

static bool _starkLoadTried;
static string _starkPkgRoot;

// X121 loading recipe: the stark editor assemblies do not sit in a
// fresh batchmode domain, so the sdk package dlls load explicitly
// (name filter tools/sdk/stark/tt) with an AssemblyResolve fallback
// before the type scan.
static Type FindLoadedType(string fullName)
{
    if (!_starkLoadTried) LoadStarkSdkAssemblies();
    foreach (var asm in AppDomain.CurrentDomain.GetAssemblies())
    {
        try
        {
            foreach (var t in asm.GetTypes())
                if (t.FullName == fullName) return t;
        }
        catch (ReflectionTypeLoadException r)
        {
            foreach (var t in r.Types)
                if (t != null && t.FullName == fullName) return t;
        }
        catch { }
    }
    return null;
}

static void LoadStarkSdkAssemblies()
{
    _starkLoadTried = true;
    string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
    string pkgRoot = Path.Combine(projectRoot, "Packages", "com.bytedance.starksdk");
    if (!Directory.Exists(pkgRoot))
    {
        string legacy = Path.Combine(projectRoot, "LocalPackages");
        if (Directory.Exists(legacy))
        {
            foreach (var d in Directory.GetDirectories(legacy))
                if (Path.GetFileName(d).IndexOf("starksdk", StringComparison.OrdinalIgnoreCase) >= 0)
                { pkgRoot = d; break; }
        }
    }
}

```

```

    }
    if (!Directory.Exists(pkgRoot)) return;
    _starkPkgRoot = pkgRoot;
    try { AppDomain.CurrentDomain.AssemblyResolve += ResolveStarkFromDomain; } catch { }
    int loaded = 0;
    foreach (var dll in Directory.GetFiles(pkgRoot, "*.dll", SearchOption.AllDirectories))
    {
        string name = Path.GetFileName(dll);
        if (name.IndexOf("tools", StringComparison.OrdinalIgnoreCase) < 0
            && name.IndexOf("sdk", StringComparison.OrdinalIgnoreCase) < 0
            && name.IndexOf("stark", StringComparison.OrdinalIgnoreCase) < 0
            && name.IndexOf("tt", StringComparison.OrdinalIgnoreCase) < 0) continue;
        try { Assembly.LoadFrom(dll); loaded++; } catch { }
    }
    Console.WriteLine("[BuildDouyin] stark assemblies loaded=" + loaded + " from " + pkgRoot);
}

static Assembly ResolveStarkFromDomain(object sender, ResolveEventArgs args)
{
    string simple = (args.Name ?? "").Split(',')[0].Trim();
    foreach (var a in AppDomain.CurrentDomain.GetAssemblies())
    {
        try { if (a.GetName().Name == simple) return a; } catch { }
    }
    try
    {
        if (_starkPkgRoot != null)
        {
            foreach (var dll in Directory.GetFiles(_starkPkgRoot, simple + ".dll", SearchOption.AllDirectories))
                return Assembly.LoadFrom(dll);
        }
    }
    catch { }
    return null;
}

// X121-proven douyin profile: real output dir, placeholder appId
// (console ids land at submission), dev build for the device batch,
// new build format (the X120 assert root cause), and web-facing
// framework/publish enums when present.
static void ApplyDouyinBuildProfile(Type settingsType, object settings, string outputDir)
{
    SetDouyinField(settings, "OutputDir", outputDir);
    SetDouyinFieldOrProperty(settings, "_appId", "tt1111111111111111");
    SetDouyinFieldOrProperty(settings, "version", "0.1.0");
    SetDouyinField(settings, "isDevBuild", true);
    SetDouyinField(settings, "isOldBuildFormat", false);
    ResolveDouyinEnumField(settings, "framework", new[] { "WebGL", "Web", "H5" });
    ResolveDouyinEnumField(settings, "_publishType", new[] { "WebGL", "Web", "MiniGame", "Game" });
    try

```

```

    {
        foreach (var m in settingsType.GetMethods(BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance))
        {
            if ((m.Name == "SaveSettings" || m.Name == "Save") && m.GetParameters().Length == 0)
            { m.Invoke(settings, null); break; }
        }
    }
    catch (Exception e) { Console.WriteLine("[BuildDouyin] settings persist failed: " + e.Message); }
    Console.WriteLine("[BuildDouyin] profile applied: OutputDir=" + outputDir);
}

static void SetDouyinField(object target, string name, object value)
{
    var f = target.GetType().GetField(name, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (f != null) f.SetValue(target, value);
}

static void SetDouyinFieldOrProperty(object target, string name, object value)
{
    var f = target.GetType().GetField(name, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (f != null) { f.SetValue(target, value); return; }
    var p = target.GetType().GetProperty(name, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (p != null && p.CanWrite) p.SetValue(target, value, null);
}

static void ResolveDouyinEnumField(object settings, string fieldName, string[] preferred)
{
    var f = settings.GetType().GetField(fieldName, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (f == null || !f.FieldType.IsEnum) return;
    string[] names = Enum.GetNames(f.FieldType);
    string chosen = null;
    foreach (var want in preferred)
    {
        foreach (var n in names)
        {
            if (string.Equals(n, want, StringComparison.OrdinalIgnoreCase)) { chosen = n; break; }
            if (chosen != null) break;
        }
        if (chosen == null)
        {
            foreach (var want in preferred)
            {
                foreach (var n in names)
                {
                    if (n.IndexOf(want, StringComparison.OrdinalIgnoreCase) >= 0) { chosen = n; break; }
                    if (chosen != null) break;
                }
            }
        }
        if (chosen != null) f.SetValue(settings, Enum.Parse(f.FieldType, chosen));
        Console.WriteLine("[BuildDouyin] enum " + fieldName + "=" + (chosen ?? "(default)"));
    }
}

```

```

static string FindDouyinPackageDir(string outputDir)
{
    if (!Directory.Exists(outputDir)) return null;
    var queue = new Queue<string>();
    queue.Enqueue(outputDir);
    while (queue.Count > 0)
    {
        string dir = queue.Dequeue();
        try
        {
            if (File.Exists(Path.Combine(dir, "game.json"))
                && File.Exists(Path.Combine(dir, "game.js"))) return dir;
            foreach (var sub in Directory.GetDirectories(dir)) queue.Enqueue(sub);
        }
        catch { }
        if (queue.Count > 64) break;
    }
    return null;
}

[Serializable]
public class CompileCheckReport
{
    public string project;
    public string unityVersion;
    public string atUtc;
    public string status;
    public int errorCount;
    public int warningCount;
    public List<string> errors;
    public List<string> warnings;
    public string exception;
}

}

// ===== FILE: Assets\_Game\Scripts\Editor\DailyAchCheck.cs =====
using System;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;
using Sy.G11.Game;

namespace Sy.G11
{
    /// <summary>
    /// T010 slice 3a machine gate: the daily quest board + the workplace-meme
    /// badge family (requirement doc S04: 3 quests per local calendar day,
    /// pure-local completion judgment, day-refresh assertion).
    /// D1 refresh_low : the same day key deals the byte-identical trio

```

```

///         twice, the three rows are always distinct, the
///         40-day walk never deals a duplicate row, and a
///         day rollover wipes the claim mask + re-arms the
///         baselines (progress counts from the new day
///         only) while re-opening the same day keeps
///         every claim exactly as it was.
/// D2 claim_law      : completion is pure progress math (baseline vs
///                   ledger); a premature claim refuses with zero
///                   state change, the target edge grants the
///                   exact reward through the real wallet/exp
///                   channels exactly once, a repeat claim refuses
///                   with zero state change, and the three mask
///                   bits stay independent.
/// D3 badge_family_law : 10 badges over unique keys with positive
///                   targets, threshold edges (target-1 locked,
///                   target unlocked), idempotent resolve, a
///                   monotone mask chain that never un-earns, the
///                   full-house mask, and every name/desc/quest
///                   key resolves from the English source.
/// D4 pure_local_law  : resolving boards/quests/badges mutates zero
///                   sim state (fingerprint unchanged), a refused
///                   claim never touches the sim, and 100 re-deals
///                   of the same day stay byte-identical.
/// D5 parity_law      : the identical 2000-second grind script (ticks +
///                   slack flips + skill upgrades + claim passes)
///                   produces a byte-identical composite (sim
///                   fingerprint + board + ledger + badge mask)
///                   on two runs, with all three quests claimed.
/// Usage: Tuanjie.exe -batchmode -quit -projectPath &lt;G11&gt; -executeMethod Sy.G11.DailyAchCheck.Run -logFile &lt;abs&gt;
/// Exit codes: 0 = all pass, 1 = assertion failure, 2 = exception.
/// </summary>
public static class DailyAchCheck
{
    public static void Run()
    {
        int code;
        try
        {
            LastFailWhere = "d1";
            bool d1 = RefreshLaw();
            string w1 = d1 ? string.Empty : LastFailWhere;
            LastFailWhere = "d2";
            bool d2 = ClaimLaw();
            string w2 = d2 ? string.Empty : LastFailWhere;
            LastFailWhere = "d3";
            bool d3 = BadgeFamilyLaw();
            string w3 = d3 ? string.Empty : LastFailWhere;
            LastFailWhere = "d4";
            bool d4 = PureLocalLaw();
            string w4 = d4 ? string.Empty : LastFailWhere;

```

```

        LastFailWhere = "d5";
        bool d5 = ParityLaw();
        string w5 = d5 ? string.Empty : LastFailWhere;
        bool all = d1 && d2 && d3 && d4 && d5;

        var sb = new StringBuilder();
        sb.AppendLine("G11 daily+badge machine gate, T010 slice 3a (requirement doc S04: 3 quests/day, pure-local judgment,
refresh assertion)");
        sb.AppendLine($"D1 refresh_law => {(d1 ? "PASS" : "FAIL at " + w1)} (same day identical trio, 3 distinct rows, 40-day
walk clean, rollover wipes claims + re-arms baselines, same-day reopen stable)");
        sb.AppendLine($"D2 claim_law => {(d2 ? "PASS" : "FAIL at " + w2)} (premature claim zero-change, edge grant exact
coin+exp once, repeat zero-change, mask bits independent)");
        sb.AppendLine($"D3 badge_family_law => {(d3 ? "PASS" : "FAIL at " + w3)} (10 badges unique keys, target edges,
idempotent + monotone resolve, full-house mask, texts resolve)");
        sb.AppendLine($"D4 pure_local_law => {(d4 ? "PASS" : "FAIL at " + w4)} (resolve mutates zero sim state, refused claim
untouched, 100 re-deals identical)");
        sb.AppendLine($"D5 parity_law => {(d5 ? "PASS" : "FAIL at " + w5)} (2000s grind script dual-run byte-identical, all
three quests claimed)");
        sb.AppendLine($"RESULT => {(all ? "ALL PASS" : "FAIL")}");

        var logsDir = Path.Combine(Path.GetFullPath(Path.Combine(Application.dataPath, "..")), "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(Path.Combine(logsDir, "g11_daily_report.txt"), sb.ToString(), new UTF8Encoding(false));

        foreach (var line in sb.ToString().Split('\n')) Debug.Log("[DailyAchCheck] " + line.TrimEnd());
        code = all ? 0 : 1;
    }
    catch (Exception e)
    {
        Debug.LogError("[DailyAchCheck] exception: " + e);
        code = 2;
    }
    EditorApplication.Exit(code);
}

static WorkerDailyAch.CareerLedger BigLedger()
{
    // every pool row completes under this ledger (claim-law helper)
    var ledger = new WorkerDailyAch.CareerLedger();
    ledger.Add(WorkerDailyAch.Stat.Slacks, 999);
    ledger.Add(WorkerDailyAch.Stat.Events, 999);
    ledger.Add(WorkerDailyAch.Stat.Skills, 99);
    ledger.Add(WorkerDailyAch.Stat.Wages, 9999);
    ledger.Add(WorkerDailyAch.Stat.Promos, 99);
    return ledger;
}

static bool SameRow(WorkerDailyAch.QuestDef a, WorkerDailyAch.QuestDef b)
=> a.Stat == b.Stat && a.Target == b.Target
&& a.RewardCoin == b.RewardCoin && a.RewardExp == b.RewardExp;

/// <summary>Step tag of the last failed law (triage face of the report).</summary>
static string LastFailWhere;

static bool Fail(string where) { LastFailWhere = where; return false; }

```

```

// ---- D1: the day-refresh law -----
static bool RefreshLaw()
{
    const string day1 = "20990921";
    const string day2 = "20990922";
    // same key deals the identical trio, twice in a row
    var t1 = WorkerDailyAch.QuestsForDay(day1);
    var t2 = WorkerDailyAch.QuestsForDay(day1);
    if (t1.Length != WorkerDailyAch.DailyQuestCount || t2.Length != WorkerDailyAch.DailyQuestCount)
        return Fail("trio_len");
    for (int i = 0; i < t1.Length; i++)
        if (!SameRow(t1[i], t2[i])) return Fail("trio_repeat");
    // the three rows are always distinct (never the same quest twice)
    if (SameRow(t1[0], t1[1]) || SameRow(t1[0], t1[2]) || SameRow(t1[1], t1[2]))
        return Fail("trio_dup:" + (int)t1[0].Stat + "/" + (int)t1[1].Stat + "/" + (int)t1[2].Stat);
    // the 40-day walk never deals a duplicate row inside any day
    for (int d = 0; d < 40; d++)
    {
        var trio = WorkerDailyAch.QuestsForDay("2099" + (900 + d).ToString("D4"));
        if (trio.Length != WorkerDailyAch.DailyQuestCount) return Fail("walk_len");
        if (SameRow(trio[0], trio[1]) || SameRow(trio[0], trio[2]) || SameRow(trio[1], trio[2]))
            return Fail("walk_dup@" + d);
    }
    // claim all three on day 1 (board armed at a zero baseline, claims
    // read the big ledger), then reopen the same day: the saved board
    // comes back with every claim intact
    var sim = new WorkerSim();
    var zeroLedger = new WorkerDailyAch.CareerLedger();
    var ledger = BigLedger();
    var b1 = WorkerDailyAch.OpenBoard(null, day1, in zeroLedger);
    for (int i = 0; i < WorkerDailyAch.DailyQuestCount; i++)
        if (WorkerDailyAch.TryClaim(b1, ledger, i, sim) != WorkerDailyAch.ClaimResult.Granted)
            return Fail("claim1@" + i + " mask=" + b1.ClaimedMask);
    string fp1 = b1.Fingerprint();
    var same = WorkerDailyAch.OpenBoard(b1, day1, in ledger);
    if (same != b1 || same.ClaimedMask != (1 << WorkerDailyAch.DailyQuestCount) - 1)
        return Fail("reopen mask=" + same.ClaimedMask);
    if (same.Fingerprint() != fp1) return Fail("reopen_fp");
    // rollover to day 2: fresh mask, baselines armed at the current
    // ledger, so progress counts only what happens after the rollover
    var b2 = WorkerDailyAch.OpenBoard(b1, day2, in ledger);
    if (b2 == b1 || b2.ClaimedMask != 0) return Fail("rollover mask=" + b2.ClaimedMask);
    if (WorkerDailyAch.Progress(b2, ledger, WorkerDailyAch.Stat.Slacks) != 0) return Fail("rollover_prog");
    if (WorkerDailyAch.TryClaim(b2, ledger, 0, sim) != WorkerDailyAch.ClaimResult.Incomplete)
        return Fail("day2_claim mask=" + b2.ClaimedMask);
    // growth after the rollover counts toward the new day; the old
    // board never moves
    var grown = ledger;
    grown.Add(WorkerDailyAch.Stat.Wages, 50);
    if (WorkerDailyAch.Progress(b2, grown, WorkerDailyAch.Stat.Wages) != 50)

```

```

        return Fail("post_roll_prog=" + WorkerDailyAch.Progress(b2, grown, WorkerDailyAch.Stat.Wages));
    if (b1.Fingerprint() != fp1) return Fail("b1_moved");
    return true;
}

// ---- D2: the claim law -----
static bool ClaimLaw()
{
    const string day = "20990921";
    var sim = new WorkerSim();
    var zero = new WorkerDailyAch.CareerLedger();
    var board = WorkerDailyAch.OpenBoard(null, day, in zero);
    var quests = WorkerDailyAch.QuestsForDay(day);
    // premature claim on zero progress: refused, zero state change
    string fpBefore = sim.StateFingerprint();
    if (WorkerDailyAch.TryClaim(board, zero, 0, sim) != WorkerDailyAch.ClaimResult.Incomplete) return false;
    if (sim.StateFingerprint() != fpBefore || board.ClaimedMask != 0) return false;
    // target edge: target-1 still refuses, target grants the exact
    // reward through the real wallet + exp channels
    var q = quests[0];
    var almost = zero;
    almost.Add(q.Stat, q.Target - 1);
    if (WorkerDailyAch.TryClaim(board, almost, 0, sim) != WorkerDailyAch.ClaimResult.Incomplete) return false;
    if (sim.StateFingerprint() != fpBefore) return false;
    var atTarget = almost;
    atTarget.Add(q.Stat, 1);
    double coinBefore = sim.Coin;
    double expBefore = sim.Exp;
    if (WorkerDailyAch.TryClaim(board, atTarget, 0, sim) != WorkerDailyAch.ClaimResult.Granted) return false;
    if (Math.Abs(sim.Coin - coinBefore - q.RewardCoin) > 1e-6) return false;
    if (Math.Abs(sim.Exp - expBefore - q.RewardExp) > 1e-6) return false;
    if (board.ClaimedMask != 1) return false;
    // repeat claim on the banked quest: refused, zero state change
    double coinAfter = sim.Coin;
    double expAfter = sim.Exp;
    string fpAfter = sim.StateFingerprint();
    if (WorkerDailyAch.TryClaim(board, atTarget, 0, sim) != WorkerDailyAch.ClaimResult.AlreadyClaimed) return false;
    if (sim.Coin != coinAfter || sim.Exp != expAfter) return false;
    if (sim.StateFingerprint() != fpAfter || board.ClaimedMask != 1) return false;
    // the other two quests stay claimable independently
    var big = BigLedger();
    if (WorkerDailyAch.TryClaim(board, big, 1, sim) != WorkerDailyAch.ClaimResult.Granted) return false;
    if (WorkerDailyAch.TryClaim(board, big, 2, sim) != WorkerDailyAch.ClaimResult.Granted) return false;
    if (board.ClaimedMask != (1 << WorkerDailyAch.DailyQuestCount) - 1) return false;
    return true;
}

// ---- D3: the badge family law -----
static bool BadgeFamilyLaw()
{

```

```

var keys = new System.Collections.Generic.HashSet<string>();
for (int i = 0; i < WorkerDailyAch.BadgeCount; i++)
{
    if (string.IsNullOrEmpty(WorkerDailyAch.BadgeKey[i])) return false;
    if (!keys.Add(WorkerDailyAch.BadgeKey[i])) return false;
    if (WorkerDailyAch.BadgeTarget[i] <= 0) return false;
}
// empty ledger earns nothing
var zero = new WorkerDailyAch.CareerLedger();
if (WorkerDailyAch.ResolveBadges(zero) != 0) return false;
// threshold edges per badge + idempotent re-resolve
for (int i = 0; i < WorkerDailyAch.BadgeCount; i++)
{
    var almost = new WorkerDailyAch.CareerLedger();
    almost.Add(WorkerDailyAch.BadgeStat[i], WorkerDailyAch.BadgeTarget[i] - 1);
    if ((WorkerDailyAch.ResolveBadges(almost) & (1 << i)) != 0) return false;
    var hit = new WorkerDailyAch.CareerLedger();
    hit.Add(WorkerDailyAch.BadgeStat[i], WorkerDailyAch.BadgeTarget[i]);
    int mask = WorkerDailyAch.ResolveBadges(hit);
    if ((mask & (1 << i)) == 0) return false;
    if (WorkerDailyAch.ResolveBadges(hit) != mask) return false;
}
// monotone chain: a growing ledger never un-earns a badge
var grow = new WorkerDailyAch.CareerLedger();
int prevMask = 0;
for (int i = 0; i < WorkerDailyAch.BadgeCount; i++)
{
    grow.Add(WorkerDailyAch.BadgeStat[i], WorkerDailyAch.BadgeTarget[i]);
    int mask = WorkerDailyAch.ResolveBadges(grow);
    if ((mask & prevMask) != prevMask) return false;
    if ((mask & (1 << i)) == 0) return false;
    prevMask = mask;
}
if (WorkerDailyAch.ResolveBadges(grow) != (1 << WorkerDailyAch.BadgeCount) - 1) return false;
// text face: every badge name/desc and every quest kind key
// resolves from the English source (never degrades to the key)
for (int i = 0; i < WorkerDailyAch.BadgeCount; i++)
{
    string name = WorkerDailyAch.BadgeName(i);
    string desc = WorkerDailyAch.BadgeDesc(i);
    if (string.IsNullOrEmpty(name) || name == WorkerDailyAch.BadgeKey[i] + "_name") return false;
    if (string.IsNullOrEmpty(desc) || desc == WorkerDailyAch.BadgeKey[i] + "_desc") return false;
}
var questStats = new[]
{
    WorkerDailyAch.Stat.Slacks, WorkerDailyAch.Stat.Events, WorkerDailyAch.Stat.Skills,
    WorkerDailyAch.Stat.Wages, WorkerDailyAch.Stat.Promos
};
foreach (var stat in questStats)
{

```

```

        string key = WorkerDailyAch.QuestKindKey(stat);
        string line = WorkerTexts.GetEn(key);
        if (string.IsNullOrEmpty(line) || line == key) return false;
    }
    return true;
}

// ---- D4: the pure-local read law -----
static bool PureLocalLaw()
{
    var sim = new WorkerSim();
    sim.GrantExp(500.0);
    for (int i = 0; i < 3; i++) sim.SetWorking(!sim.Working);
    string fp = sim.StateFingerprint();
    // resolving boards, quests and badges is read-only over the sim
    var ledger = new WorkerDailyAch.CareerLedger();
    ledger.Add(WorkerDailyAch.Stat.Slacks, 3);
    ledger.Add(WorkerDailyAch.Stat.Wages, 42);
    var board = WorkerDailyAch.OpenBoard(null, "20990921", in ledger);
    for (int i = 0; i < WorkerDailyAch.DailyQuestCount; i++)
    {
        WorkerDailyAch.IsComplete(board, ledger, i);
        WorkerDailyAch.QuestLine(board, ledger, i);
    }
    WorkerDailyAch.ResolveBadges(ledger);
    if (sim.StateFingerprint() != fp) return false;
    // a refused claim never touches the sim (baseline == ledger, so
    // every quest reads zero progress and must refuse)
    if (WorkerDailyAch.TryClaim(board, ledger, 0, sim) == WorkerDailyAch.ClaimResult.Granted) return false;
    if (sim.StateFingerprint() != fp) return false;
    // the deal chain is stable: 100 re-deals of the same day stay identical
    var first = WorkerDailyAch.QuestsForDay("20260921");
    for (int i = 0; i < 100; i++)
    {
        var again = WorkerDailyAch.QuestsForDay("20260921");
        for (int q = 0; q < first.Length; q++)
            if (!SameRow(first[q], again[q])) return false;
    }
    return true;
}

// ---- D5: the dual-run byte-parity law -----
// Script-day pick: the grind script can deterministically complete
// slacks (flips), skills (upgrades) and wages (beats) quests, but the
// events/promos rows depend on the flow's card cadence - so the script
// day is picked (fixed candidate order, zero randomness) as the first
// day whose trio stays inside the script-reachable stats.
static readonly string[] ScriptDayCandidates =
{
    "20990921", "20990922", "20990923", "20990924", "20990925",

```

```

        "20990926", "20990927", "20990928", "20990929", "20990930",
        "20991001", "20991002", "20991003", "20991004", "20991005",
        "20991006", "20991007", "20991008", "20991009", "20991010"
    };

    static string PickScriptDay()
    {
        foreach (var day in ScriptDayCandidates)
        {
            var trio = WorkerDailyAch.QuestsForDay(day);
            bool ok = true;
            for (int i = 0; i < trio.Length; i++)
            {
                var s = trio[i].Stat;
                if (s != WorkerDailyAch.Stat.Slacks && s != WorkerDailyAch.Stat.Skills
                    && s != WorkerDailyAch.Stat.Wages) ok = false;
            }
            if (ok) return day;
        }
        return ScriptDayCandidates[0];
    }

    static bool ParityLaw()
    {
        string day = PickScriptDay();
        string a = RunGrindScript(day);
        string b = RunGrindScript(day);
        if (a != b) return Fail("parity");
        if (!a.Contains("|granted:3")) return Fail("granted " + a.Substring(a.IndexOf("|trio:", StringComparison.Ordinal))); //
all three quests banked
        if (!a.Contains("|claims:7")) return Fail("claims"); // full claim mask
        if (!a.Contains("|badges:")) return Fail("badges_tag");
        // the grind surely crossed the first-wage, pie-survivor and
        // toilet-master thresholds (2000+ wage beats, 190+ slack flips,
        // the 10s opener card survived)
        int mask = ParseMask(a);
        if ((mask & 0x13) != 0x13) return Fail("mask=" + mask); // bits 0,1,4
        return true;
    }

    static int ParseMask(string composite)
    {
        int at = composite.IndexOf("|badges:", StringComparison.Ordinal);
        if (at < 0) return -1;
        int start = at + 8;
        int end = composite.IndexOf('|', start);
        string digits = end < 0 ? composite.Substring(start) : composite.Substring(start, end - start);
        return int.Parse(digits, System.Globalization.CultureInfo.InvariantCulture);
    }

    /// <summary>A 2000-second deterministic grind on the picked script day:

```

```

/// wage beats 1/s, a slack flip every 10s, an upfront exp grant funds
/// three PPT upgrades, and a claim pass runs every tick. The picked
/// trio stays inside the script-reachable stats, so all three quests
/// are guaranteed to complete inside the span. The lifetime ledger
/// mirrors the sim counters, exactly like the 3b host will.</summary>
static string RunGrindScript(string day)
{
    var sim = new WorkerSim();
    sim.GrantExp(1500.0f);
    var ledger = new WorkerDailyAch.CareerLedger();
    var board = WorkerDailyAch.OpenBoard(null, day, in ledger);
    var trio = WorkerDailyAch.QuestsForDay(day);
    int granted = 0;
    float t = 0f;
    const float dt = 0.1f;
    const float span = 2000f;
    while (t < span)
    {
        sim.Tick(dt);
        float prev = t;
        t += dt;
        if ((int)(t / 10f) != (int)(prev / 10f)) sim.SetWorking(!sim.Working);
        // the three affordable upgrades: skills quest target is 3
        while (sim.SkillLevel(WorkerData.SkillBranch.Ppt) < WorkerData.SkillLevelCount
            && sim.SkillUpgradeCost(WorkerData.SkillBranch.Ppt) > 0
            && sim.Exp >= sim.SkillUpgradeCost(WorkerData.SkillBranch.Ppt))
            sim.TryUpgradeSkill(WorkerData.SkillBranch.Ppt);
        ledger.Wages = sim.WageBeatCount;
        ledger.Events = sim.EventLog.Count;
        ledger.Skills = sim.SkillUpgradeCount;
        ledger.Slacks = sim.SlackSessions;
        ledger.Promos = sim.PromoteCount;
        for (int i = 0; i < WorkerDailyAch.DailyQuestCount; i++)
            if (WorkerDailyAch.TryClaim(board, ledger, i, sim) == WorkerDailyAch.ClaimResult.Granted)
                granted++;
    }
    int mask = WorkerDailyAch.ResolveBadges(ledger);
    return sim.StateFingerprint() + "|" + board.Fingerprint() + "|" + ledger.Fingerprint()
        + "|granted:" + granted + "|claims:" + board.ClaimedMask + "|badges:" + mask
        + "|trio:" + (int)trio[0].Stat + "," + (int)trio[1].Stat + "," + (int)trio[2].Stat
        + "|ev:" + sim.EventLog.Count + "|beats:" + sim.WageBeatCount;
}
}
}
// ===== FILE: Assets\_Game\Scripts\Editor\DataSchemaCheck.cs =====
using System;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Text;

```

```

using Sy.Core;
using Sy.G11.Game;
using UnityEditor;
using UnityEngine;

namespace Sy.G11
{
    /// <summary>
    /// T004 schema gate (batchmode entry):
    /// Tuanjie.exe -batchmode -quit -projectPath &lt;p>&gt; -executeMethod Sy.G11.DataSchemaCheck.Run -logFile &lt;abs>&gt;
    /// Exit codes: 0 = all 13 laws green, 1 = at least one law failed.
    /// The 13 laws map 1:1 to G11_鐳板€煎寔.md section 9 (schema freeze table).
    /// Const fields are read into locals before comparison so the compiler cannot
    /// fold the guards into unreachable code (zero-warning discipline).
    /// </summary>
    public static class DataSchemaCheck
    {
        public static void Run()
        {
            var laws = new Laws();
            int fails = 0;
            void Step(string name, Func<bool> body)
            {
                bool ok;
                string note = "";
                try
                {
                    ok = body();
                }
                catch (Exception e)
                {
                    ok = false;
                    note = " exception=" + e.Message;
                }
                Debug.Log("[DataSchemaCheck] " + name + " " + (ok ? "GREEN" : "FAIL") + note);
                if (!ok) fails++;
            }

            Step("wage_schema_law", laws.WageSchema);
            Step("dual_axis_law", laws.DualAxis);
            Step("skill_table_law", laws.SkillTable);
            Step("promotion_law", laws.Promotion);
            Step("event_table_law", laws.EventTable);
            Step("offline_law", laws.Offline);
            Step("adcap_law", laws.AdCap);
            Step("inherit_law", laws.Inherit);
            Step("signature_freeze_law", laws.Signature);
            Step("texts_en_law", laws.TextsEn);
            Step("texts_zh_law", laws.TextsZh);
            Step("zero_scattered_copy_law", laws.ZeroScatteredCopy);
        }
    }
}

```

```

Step("placeholder_law", laws.Placeholder);

var projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
var logsDir = Path.Combine(projectRoot, "Logs");
if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
string sig = WorkerData.DeriveSigA();
string fingerprint = "len=" + sig.Length + " fnv=" + Fnv(sig).ToString("X8");
string summary = (fails == 0 ? "ALL_GREEN 13/13" : "FAILED " + fails) + " sig " + fingerprint;
var sb = new StringBuilder();
sb.AppendLine("[DataSchemaCheck] " + summary);
sb.AppendLine("[DataSchemaCheck] sig_text=" + sig);
File.WriteAllText(Path.Combine(logsDir, "dataschema_report.txt"), sb.ToString(), new UTF8Encoding(false));
Debug.Log("[DataSchemaCheck] " + summary + " -> Logs/dataschema_report.txt");
EditorApplication.Exit(fails == 0 ? 0 : 1);
}

static uint Fnv(string s)
{
    uint h = 2166136261u;
    for (int i = 0; i < s.Length; i++)
    {
        h ^= s[i];
        h *= 16777619u;
    }
    return h;
}

static bool IsCjk(char c) => c >= 0x2E80;

static bool Monotonic(float[] a)
{
    for (int i = 1; i < a.Length; i++)
        if (a[i] < a[i - 1]) return false;
    return true;
}

static bool Monotonic(int[] a)
{
    for (int i = 1; i < a.Length; i++)
        if (a[i] < a[i - 1]) return false;
    return true;
}

sealed class Laws
{
    public bool WageSchema()
    {
        float baseWage = WorkerData.BaseWagePerSec;
        float expPerSec = WorkerData.ExpPerSecOnline;
        long coinCap = WorkerData.CoinCap;
    }
}

```

```

    long expCap = WorkerData.ExpCap;
    if (baseWage != 1.0f) return false;
    if (expPerSec != 1.0f) return false;
    if (coinCap <= 0 || expCap <= 0) return false;
    if (WorkerData.RankCoef.Length != 5) return false;
    if (WorkerData.RankCoef[0] != 1.0f) return false;
    if (!Monotonic(WorkerData.RankCoef)) return false;
    // four-factor chain sanity at neutral state
    if (Math.Abs(WorkerData.WagePerSec(0, 0, 0) - 1.0f) > 1e-6f) return false;
    if (Math.Abs(WorkerData.WagePerSec(4, 3, 0) - 7.0f * 2.2f) > 1e-4f) return false;
    return true;
}

public bool DualAxis()
{
    float workKpi = WorkerData.WorkKpiPerSec;
    float workMood = WorkerData.WorkMoodPerSec;
    float slackKpi = WorkerData.SlackKpiPerSec;
    float slackMood = WorkerData.SlackMoodPerSec;
    float slackInsp = WorkerData.SlackInspPerSec;
    float initKpi = WorkerData.InitKpi;
    float initMood = WorkerData.InitMood;
    float initInsp = WorkerData.InitInsp;
    float axisMin = WorkerData.AxisMin;
    float axisMax = WorkerData.AxisMax;
    float moodStrike = WorkerData.MoodStrikeLine;
    float moodRecoverTo = WorkerData.MoodRecoverTo;
    float moodRecoverRate = WorkerData.MoodRecoverPerSec;
    float kpiWarn = WorkerData.KpiWarnLine;
    float kpiRecoverSec = WorkerData.KpiRecoverWorkSeconds;
    float rareGate = WorkerData.RarePoolInspGate;
    if (!(workKpi > 0) || !(workMood < 0)) return false;
    if (!(slackKpi < 0) || !(slackMood > 0) || !(slackInsp > 0)) return false;
    if (initKpi != 40f || initMood != 60f || initInsp != 0f) return false;
    if (axisMin != 0f || axisMax != 100f) return false;
    if (moodStrike != 20f || moodRecoverTo != 25f) return false;
    if (!(moodRecoverRate > 0f)) return false;
    if (kpiWarn != 20f || kpiRecoverSec != 10f) return false;
    if (rareGate != 30f) return false;
    // helper faces: working raises KPI, slacking raises mood (skill mults engaged)
    if (Math.Abs(WorkerData.KpiPerSec(true, 0, 0) - 2.0f) > 1e-6f) return false;
    if (Math.Abs(WorkerData.KpiPerSec(true, 1, 0) - 2.2f) > 1e-5f) return false;
    if (Math.Abs(WorkerData.MoodPerSec(false, 0) - 2.0f) > 1e-6f) return false;
    if (Math.Abs(WorkerData.MoodPerSec(false, 1) - 2.0f * 1.30f) > 1e-5f) return false;
    if (Math.Abs(WorkerData.MoodPerSec(false, 3) - 2.0f * 2.50f) > 1e-5f) return false;
    // SlackArt L3 halves the KPI loss
    if (Math.Abs(WorkerData.KpiPerSec(false, 0, 3) - (-0.25f)) > 1e-6f) return false;
    return true;
}

```

(middle pages omitted)

pages 31 - 188 of 218 omitted per first-30 + last-30 excerpt rule.

```

        if (Consumed || !Claimable || Gain <= 0.0) return -1L;
        double deposit = doubled ? Gain * 2.0 : Gain;
        long granted = (long)Math.Round(deposit);
        if (sim != null) sim.DepositCoin(deposit);
        Stamp(nowMs);
        Consumed = true;
        Track("granted");
        return granted;
    }

    /// <summary>Once-per-span double offer resolution (numeric doc section
    /// 6, rewarded_double 5/day): pre-offer category gate first (the honest
    /// hint instead of a dead ad call), then the ad verdict, then the local
    /// cap consume - and only a grant burns the span's offer and banks the
    /// x2 through the real wallet. Refused paths change zero claim state;
    /// a blocked or failed offer leaves the claim collectible as-is.</summary>
    public DoubleResult TryDouble(AdCapPolicy policy, string dayKey, bool adWatched, WorkerSim sim, long nowMs)
    {
        if (Consumed || !Claimable || Gain <= 0.0) return DoubleResult.Refused;
        if (DoubleOfferUsed) return DoubleResult.Refused;
        Track("accept");
        string category = AdCategories.CategoryOfPoint(PointId);
        if (policy != null && !policy.HasCategoryBudget(dayKey, category))
        {
            Track("blocked");
            return DoubleResult.Blocked;
        }
        if (!adWatched)
        {
            Track("ad_fail");
            return DoubleResult.AdFail;
        }
        if (policy != null && !policy.TryConsumeRewarded(dayKey, category))
        {
            Track("blocked");
            return DoubleResult.Blocked;
        }
        DoubleOfferUsed = true;
        TryCollect(true, sim, nowMs);
        return DoubleResult.Granted;
    }
}

}

}

}

}

// ===== FILE: Assets\_Game\Scripts\Runtime\Game\WorkerPlaceholder.cs =====
using System;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;

```

```

namespace Sy.G11.Game
{
    /// <summary>
    /// Procedural placeholder faces (X027 two-state law): the art catalog stem wins
    /// when real art exists, the deterministic procedural face is the fallback.
    /// Six faces cover the S1 slice: intern unit, four office states, event card frame.
    /// Zero RNG: every pixel is a pure function of (stem, size, palette), so any two
    /// builds are byte-identical and the schema check can assert determinism.
    /// </summary>
    public static class WorkerPlaceholder
    {
        // Stems match the art requirement doc section 7 work orders (A01/A06-A09/A15).
        public const string StemIntern = "G11_unit_01_intern";
        public const string StemOfficeOpen = "G11_scene_office_open";
        public const string StemOfficeCubicle = "G11_scene_office_cubicle";
        public const string StemOfficePrivate = "G11_scene_office_private";
        public const string StemOfficePenthouse = "G11_scene_office_penthouse";
        public const string StemEventCardFrame = "G11_event_card_frame";

        // T016 P-slice: the remaining nine S1 work-order stems (A02-A05, A10-A14).
        // Badge-tiers (A13) and skills (A14) are multi-cell sheets - the view
        // addresses sliced cells as stem + "_" + index (badge 0..4 by rank,
        // skill 0..3 by branch order Ppt/Report/SlackArt/ManageUp), the stems
        // register now so the catalog keys are final before any art arrives.
        public const string StemUnitBossWang = "G11_unit_02_boss_wang";
        public const string StemUnitHrTian = "G11_unit_03_hr_tian";
        public const string StemUnitKevin = "G11_unit_04_kevin";
        public const string StemUnitMasterZhang = "G11_unit_05_master_zhang";
        public const string StemIconCoin = "G11_icon_ui_coin";
        public const string StemIconMood = "G11_icon_ui_mood";
        public const string StemIconKpi = "G11_icon_ui_kpi";
        public const string StemIconBadgeTiers = "G11_icon_badge_tiers";
        public const string StemIconSkills = "G11_icon_skills";

        /// <summary>Series accent: electric purple (design language anchor).</summary>
        public static readonly Color32 AccentElectro = new Color32(0x8B, 0x6E, 0xF0, 0xFF);

        static readonly Color32 BaseSlate = new Color32(0x2E, 0x33, 0x40, 0xFF);
        static readonly Color32 BaseCubicle = new Color32(0x3A, 0x40, 0x4C, 0xFF);
        static readonly Color32 BasePrivate = new Color32(0x46, 0x4C, 0x5A, 0xFF);
        static readonly Color32 BasePenthouse = new Color32(0x1C, 0x20, 0x2E, 0xFF);
        static readonly Color32 BaseCard = new Color32(0x24, 0x28, 0x34, 0xFF);
        // T016 P-slice placeholder palettes: one distinct base per new stem so the
        // procedural fallback stays visually distinguishable before D-tier art lands.
        static readonly Color32 BaseBoss = new Color32(0x3D, 0x33, 0x52, 0xFF);
        static readonly Color32 BaseHr = new Color32(0x4A, 0x2E, 0x3C, 0xFF);
        static readonly Color32 BaseKevin = new Color32(0x33, 0x3C, 0x33, 0xFF);
        static readonly Color32 BaseMaster = new Color32(0x2C, 0x46, 0x40, 0xFF);
        static readonly Color32 BaseCoin = new Color32(0x54, 0x44, 0x20, 0xFF);
        static readonly Color32 BaseMood = new Color32(0x40, 0x33, 0x4C, 0xFF);
    }
}

```

```

static readonly Color32 BaseKpi = new Color32(0x48, 0x2E, 0x2C, 0xFF);
static readonly Color32 BaseBadge = new Color32(0x50, 0x44, 0x2C, 0xFF);
static readonly Color32 BaseSkill = new Color32(0x2E, 0x3D, 0x52, 0xFF);

static readonly Dictionary<string, Sprite> Cache = new Dictionary<string, Sprite>(StringComparer.Ordinal);

/// <summary>Two-state resolve: catalog sprite first, procedural fallback after (null-safe).</summary>
public static Sprite Face(string stem, int size, Color32 baseCol)
{
    lock (Cache)
    {
        if (Cache.TryGetValue(stem, out var cached)) return cached;
    }
    var catalogSprite = ArtRuntime.Get(stem);
    Sprite sprite = catalogSprite != null ? catalogSprite : BuildSprite(Procedural(stem, size, baseCol));
    lock (Cache)
    {
        Cache[stem] = sprite;
    }
    return sprite;
}

/// <summary>Deterministic procedural texture: checker tiles, hash-driven accent stripes, accent border.</summary>
public static Texture2D Procedural(string stem, int size, Color32 baseCol)
{
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    tex.filterMode = FilterMode.Point;
    tex.wrapMode = TextureWrapMode.Clamp;
    uint h = StemHash(stem);
    int stripeStep = 4 + (int)(h % 5u); // 4..8 px accent stripe cadence
    int stripeOffset = (int)((h >> 8) % 8u);
    int tile = 8; // checker tile size
    var px = new Color32[size * size];
    for (int y = 0; y < size; y++)
    {
        for (int x = 0; x < size; x++)
        {
            var c = (((x / tile) + (y / tile)) & 1) == 0 ? baseCol : Shade(baseCol, 18);
            bool border = x < 2 || y < 2 || x >= size - 2 || y >= size - 2;
            bool stripe = ((x + stripeOffset) % stripeStep) == 0 && y < size / 4;
            if (border) c = AccentElectro;
            else if (stripe) c = Shade(AccentElectro, -40);
            px[y * size + x] = c;
        }
    }
    tex.SetPixels32(px);
    tex.Apply(false, false);
    return tex;
}

```

```

/// <summary>All six face stems, in doc order, for the schema check sweep.</summary>
public static string[] AllFaceStems() => new[]
{
    StemIntern, StemOfficeOpen, StemOfficeCubicle, StemOfficePrivate, StemOfficePenthouse, StemEventCardFrame
};

/// <summary>T016: the full fifteen S1 work-order stems in doc order A01-A15
/// (art requirement doc section 7). The frozen six-face sweep above stays
/// untouched; this list is the T016 wiring surface for ArtWireCheck.</summary>
public static string[] AllT016Stems() => new[]
{
    StemIntern, StemUnitBossWang, StemUnitHrTian, StemUnitKevin, StemUnitMasterZhang,
    StemOfficeOpen, StemOfficeCubicle, StemOfficePrivate, StemOfficePenthouse,
    StemIconCoin, StemIconMood, StemIconKpi, StemIconBadgeTiers, StemIconSkills,
    StemEventCardFrame
};

/// <summary>Base color per face (office states get progressively darker/tidier palettes).</summary>
public static Color32 BaseColorFor(string stem)
{
    if (stem == StemOfficeOpen) return BaseSlate;
    if (stem == StemOfficeCubicle) return BaseCubicle;
    if (stem == StemOfficePrivate) return BasePrivate;
    if (stem == StemOfficePenthouse) return BasePenthouse;
    if (stem == StemEventCardFrame) return BaseCard;
    if (stem == StemUnitBossWang) return BaseBoss;
    if (stem == StemUnitHrTian) return BaseHr;
    if (stem == StemUnitKevin) return BaseKevin;
    if (stem == StemUnitMasterZhang) return BaseMaster;
    if (stem == StemIconCoin) return BaseCoin;
    if (stem == StemIconMood) return BaseMood;
    if (stem == StemIconKpi) return BaseKpi;
    if (stem == StemIconBadgeTiers) return BaseBadge;
    if (stem == StemIconSkills) return BaseSkill;
    return BaseSlate;
}

static Sprite BuildSprite(Texture2D tex)
    => Sprite.Create(tex, new Rect(0, 0, tex.width, tex.height), new Vector2(0.5f, 0.5f), 100f, 0, SpriteMeshType.FullRect);

static Color32 Shade(Color32 c, int d)
    => new Color32(ClampByte(c.r + d), ClampByte(c.g + d), ClampByte(c.b + d), c.a);

static byte ClampByte(int v) => (byte)Mathf.Clamp(v, 0, 255);

static uint StemHash(string stem)
{
    uint h = 2166136261u;
    for (int i = 0; i < stem.Length; i++)
    {

```

```

        h ^= stem[i];
        h *= 16777619u;
    }
    return h;
}
}
}
}
// ===== FILE: Assets\Game\Scripts\Runtime\Game\WorkerSaveData.cs =====
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using UnityEngine;

namespace Sy.G11.Game
{
    /// <summary>
    /// T012 data-layer frozen mirror of the local career save (red line 12: pure
    /// local authority, zero server). Field face = requirements doc section 10:
    /// coin / exp / rank / kpi / mood / insp / 4 skill levels / events-seen set /
    /// offline timestamp / NG+ inheritance slot (ngLoop + coin-keep law). Two
    /// determinism carries (simTime + wageBeatCarry) ride along so a resumed run
    /// continues the same beat/op stream (resume-the-journey law).
    /// Host-side saves (g11_gacha_* / g11_career_*) stay in PlayerPrefs and merge
    /// through a READ-ONLY loader with the Host wiring slice - this data layer
    /// owns zero PlayerPrefs write paths (the load zero-overwrite guard shape).
    /// Corrupt or schema-mismatched JSON refuses to load and answers the
    /// fresh-start face: never a crash, never a partial apply.
    /// </summary>
    [Serializable]
    public sealed class WorkerSaveData
    {
        public const int MirrorSchemaVersion = 1;

        public int schemaVersion = MirrorSchemaVersion;
        public double coin;           // wallet (survives the NG+ retire: coin-keep law)
        public double exp;            // skill currency
        public int rankIndex;         // career rank
        public int ngloop;           // NG+ inheritance slot (wage x(1+0.10/loop))
        public int retireCount;       // retire history (carries across loops)
        public float kpi;             // dual axis 1
        public float mood;           // dual axis 2
        public float insp;           // inspiration axis
        public bool working = true;   // work/slack mode
        public double simTime;        // determinism carry: dt-stream resume point
        public double wageBeatCarry;  // determinism carry: settle-axis beat carry
        public int[] skillLevels = new int[4];
        public string[] eventsSeen = Array.Empty<string>();
        public long offlineUnix;     // wall-clock stamp (offline settle anchor)
    }
}

```

```

/// <summary>Capture the frozen mirror off a live sim. Single source: the
/// sim owns the whole live face; the distinct resolved event ids fold the
/// events-seen set out of the flow log.</summary>
public static WorkerSaveData Capture(WorkerSim sim, long offlineUnix)
{
    var save = new WorkerSaveData
    {
        schemaVersion = MirrorSchemaVersion,
        coin = sim.Coin,
        exp = sim.Exp,
        rankIndex = sim.RankIndex,
        ngLoop = sim.NgLoop,
        retireCount = sim.RetireCount,
        kpi = sim.Kpi,
        mood = sim.Mood,
        insp = sim.Insp,
        working = sim.Working,
        simTime = sim.SimTime,
        wageBeatCarry = sim.WageBeatCarry,
        skillLevels = new[]
        {
            sim.SkillLevel(WorkerData.SkillBranch.Ppt),
            sim.SkillLevel(WorkerData.SkillBranch.Report),
            sim.SkillLevel(WorkerData.SkillBranch.SlackArt),
            sim.SkillLevel(WorkerData.SkillBranch.ManageUp)
        },
        offlineUnix = offlineUnix
    };
    var seen = new List<string>();
    var log = sim.EventLog;
    for (int i = 0; i < log.Count; i++)
    {
        string id = log[i].Id;
        if (!seen.Contains(id)) seen.Add(id);
    }
    save.eventsSeen = seen.ToArray();
    return save;
}

public string ToJson() => JsonUtility.ToJson(this);

/// <summary>Load law: corrupt / empty / schema-mismatched JSON answers the
/// fresh-start face (never null, never throws to the caller).</summary>
public static WorkerSaveData FromJsonOrFresh(string json)
{
    if (!string.IsNullOrEmpty(json))
    {
        try
        {
            var save = JsonUtility.FromJson<WorkerSaveData>(json);

```

```

        if (save != null && save.schemaVersion == MirrorSchemaVersion
            && save.skillLevels != null && save.skillLevels.Length == 4)
            return save;
    }
    catch (System.Exception) { /* corrupt payload: fall through to fresh */ }
}
return FreshStart();
}

/// <summary>The fresh-start face: identical to a brand-new WorkerSim
/// constructor face (init axes, wallet empty, intern rank, loop zero).</summary>
public static WorkerSaveData FreshStart()
{
    return new WorkerSaveData
    {
        kpi = WorkerData.InitKpi,
        mood = WorkerData.InitMood,
        insp = WorkerData.InitInsp,
        working = true,
        skillLevels = new int[4],
        eventsSeen = Array.Empty<string>()
    };
}

/// <summary>Full-mirror round-trip fingerprint (field-by-field canary).</summary>
public string FaceFingerprint()
{
    var sb = new StringBuilder();
    sb.Append("v").Append(schemaVersion).Append(';');
    sb.Append("coin:").Append(coin.ToString("R", Inv)).Append(';');
    sb.Append("exp:").Append(exp.ToString("R", Inv)).Append(';');
    sb.Append("rank:").Append(rankIndex).Append(",ng:").Append(ngLoop)
        .Append(",ret:").Append(retireCount).Append(';');
    sb.Append("axis:").Append(kpi.ToString("R", Inv)).Append(',')
        .Append(mood.ToString("R", Inv)).Append(',')
        .Append(insp.ToString("R", Inv)).Append(';');
    sb.Append("mode:").Append(working ? 1 : 0).Append(';');
    sb.Append("t:").Append(simTime.ToString("R", Inv)).Append(',')
        .Append(wageBeatCarry.ToString("R", Inv)).Append(';');
    sb.Append("skills:").Append(SkillsText()).Append(';');
    sb.Append("seen:").Append(SeenText()).Append(';');
    sb.Append("off:").Append(offlineUnix);
    return sb.ToString();
}

/// <summary>Core-face fingerprint (mirror minus the events-seen fold): the
/// restore idempotence canary - the flow slate re-arms fresh after a
/// restore, so the seen set is a point-in-time fold, not a restored face.</summary>
public string CoreFaceFingerprint()
{

```

```

        var sb = new StringBuilder();
        sb.Append("v").Append(schemaVersion).Append(';');
        sb.Append("coin:").Append(coin.ToString("R", Inv)).Append(';');
        sb.Append("exp:").Append(exp.ToString("R", Inv)).Append(';');
        sb.Append("rank:").Append(rankIndex).Append(",ng:").Append(ngLoop)
            .Append(",ret:").Append(retireCount).Append(';');
        sb.Append("axis:").Append(kpi.ToString("R", Inv)).Append(',')
            .Append(mood.ToString("R", Inv)).Append(',')
            .Append(insp.ToString("R", Inv)).Append(';');
        sb.Append("mode:").Append(working ? 1 : 0).Append(';');
        sb.Append("t:").Append(simTime.ToString("R", Inv)).Append(',')
            .Append(wageBeatCarry.ToString("R", Inv)).Append(';');
        sb.Append("skills:").Append(SkillsText()).Append(';');
        sb.Append("off:").Append(offlineUnix);
        return sb.ToString();
    }

    string SkillsText()
        => skillLevels == null ? "null" : string.Join(",", skillLevels);

    string SeenText()
        => eventsSeen == null ? "null" : string.Join("|", eventsSeen);

    static readonly CultureInfo Inv = CultureInfo.InvariantCulture;
}
// ===== FILE: Assets\\Game\\Scripts\\Runtime\\Game\\WorkerSaveWire.cs =====
using UnityEngine;

namespace Sy.G11.Game
{
    /// <summary>
    /// T012 slice 2 host wiring: the unified save slot (red line 12: pure local
    /// authority, zero server). The frozen mirror round-trips through exactly
    /// one PlayerPrefs key; the host-side faces (g11_gacha_* / g11_career_*)
    /// stay outside this wiring - the load zero-overwrite guard: this class
    /// never writes another face's keys, so a resume can never clobber the
    /// gacha chain or the career ledger.
    /// Save bookends (quit / destroy / pause / promote / retire) flush through
    /// SaveSlot; Awake discriminates a returning player through HasRealSave (a
    /// wall-clock stamp only a real save carries) so the day-one tutorial arms
    /// for new players only.
    /// </summary>
    public static class WorkerSaveWire
    {
        const string MirrorPrefsKey = "g11_save_mirror";

        /// <summary>Unified load: the mirror JSON answers the fresh-start face
        /// on a missing or corrupt slot (never null, never throws).</summary>
        public static WorkerSaveData LoadSlot()

```

```

=> WorkerSaveData.FromJsonOrFresh(PlayerPrefs.GetString(MirrorPrefsKey, ""));

/// <summary>Unified save: one capture, one key, one flush.</summary>
public static void SaveSlot(WorkerSim sim)
{
    var save = WorkerSaveData.Capture(sim, WorkerOffline.NowMs());
    PlayerPrefs.SetString(MirrorPrefsKey, save.ToJson());
    PlayerPrefs.Save();
}

/// <summary>Returning-player discrimination: only a real save carries a
/// wall-clock offline stamp (the fresh face ships offlineUnix = 0), so
/// the tutorial arms exactly once - on the true first run.</summary>
public static bool HasRealSave(WorkerSaveData save)
    => save != null && save.offlineUnix > 0L;

/// <summary>Test probe: wipes the mirror slot only (sentinels and the
/// host-side faces belong to their callers).</summary>
public static void ClearSlotForTests()
{
    PlayerPrefs.DeleteKey(MirrorPrefsKey);
    PlayerPrefs.Save();
}
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Game\WorkerShareSpecs.cs =====
using System;
using System.Globalization;

namespace Sy.G11.Game
{
    /// <summary>
    /// T010 slice 2: the auto-composed show-off card specs (doc section 8: the
    /// offline summary data - away span / paid coins / missed events - is the
    /// brag text source). Win-only law, series-isomorphic with the P02 share
    /// specs: an unclaimable or empty claim never brags (null), a claimable
    /// claim brags the frozen numbers - never a fabricated figure. The card
    /// face stays procedural until the art batch lands a template stem
    /// (template key null walks the ShareCardService fallback bg, headless
    /// green); the compose budget ladder and the pixel-readback canary live
    /// in the shared Sy.Social service and ShareCardCheck.
    /// </summary>
    public static class WorkerShareSpecs
    {
        public const string Footer = "Crazy Worker - Idle Office Life";
        public const string OfflineTitleKey = "share_offline_title";
        public const string OfflineCoinsKey = "share_offline_line_coins";
        public const string OfflineEventsKey = "share_offline_line_events";
        public const string OfflineHookKey = "share_offline_line_hook";
    }
}

```

```

    /// <summary>Brag spec from an open offline claim, or null when the claim
    /// never reached the one-hour threshold or carries no gain (a short
    /// visit is not a brag). Deterministic: the same claim composes the
    /// same copy byte-for-byte on every call.</summary>
    public static Sy.Social.ShareCard FromOfflineClaim(WorkerOffline.OfflineClaim claim)
    {
        if (claim == null || !claim.Claimable || claim.Gain <= 0.0) return null;
        long minutes = (long)Math.Round(claim.AwaySec / 60.0);
        long coins = (long)Math.Round(claim.Gain);
        return Sy.Social.AutoCardComposer.Card(null,
            string.Format(CultureInfo.InvariantCulture, WorkerTexts.GetEn(OfflineTitleKey), minutes),
            new[]
            {
                string.Format(CultureInfo.InvariantCulture, WorkerTexts.GetEn(OfflineCoinsKey), coins),
                string.Format(CultureInfo.InvariantCulture, WorkerTexts.GetEn(OfflineEventsKey), claim.MissedEvents),
                WorkerTexts.GetEn(OfflineHookKey)
            },
            Footer,
            "src=share&span=" + minutes.ToString(CultureInfo.InvariantCulture));
    }
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Game\WorkerSim.cs =====
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Security.Cryptography;
using System.Text;
using Sy.Platform;

namespace Sy.G11.Game
{
    /// <summary>
    /// T005 wage main axis simulation core (pure C#, zero RNG, no scene dependency).
    /// Determinism law: state is a pure function of the (dt stream, op stream), so any
    /// two runs with identical inputs produce byte-identical state fingerprints -
    /// which is the offline/online isomorphism law of the numeric doc section 1.
    /// Red line 11 anchors owned by this slice: hook content live from frame zero,
    /// first wage beat lands within 1.5s with zero player ops (zero login wall),
    /// the beat carries the number+pulse+sound three-sync contract, and the Work/Slack
    /// toggle answers from t=0 with no unlock gate in front.
    /// T006-a extension: the sim hosts the SYS04 WorkerEventFlow - it feeds the flow
    /// a per-tick state view, applies the resolved card fx (clamped axes), owns the
    /// forced-rest freeze window (evt_07) and folds the flow EventSig into the state
    /// fingerprint so the dual-run byte-parity law covers the event face too.
    /// </summary>
    public sealed class WorkerSim
    {
        /// <summary>Wage settle cadence: one beat per second (numeric doc section 1, W01).</summary>
        public const float WageBeatPeriodSec = 1f;
    }
}

```

```

// ----- T007 comedy skill tree (doc section 3): the red-line-11 first-skill beat -----
/// <summary>Tutorial guide slot: inside the T+3-10s window of the per-second table.
/// The day-one red packet carries exactly one L1 cost (100 exp, doc section 3),
/// so a fresh hire can land the first skill upgrade inside the 8s bound.</summary>
public const float TutorialGuideAtSec = 5f;
public const double TutorialRedPacketExp = 100.0;

/// <summary>Upgrade verdict for the skill face: Ok / NoExp (refused, zero state
/// change) / Maxed (level cap reached, zero state change).</summary>
public enum SkillUpResult { Ok = 0, NoExp = 1, Maxed = 2 }

// ----- live state -----
double _coin;
double _exp;
double _simTime;
double _wageBeatCarry;
float _kpi;
float _mood;
float _insp;
bool _working = true;
int _rankIndex;
int _ngLoop;
readonly int[] _skillLevels = new int[4];

// ----- T008: dual-axis flip shows + promotion gates + NG+ endings -----
bool _strike;
int _strikeCount;
bool _warn;
int _warnCount;
float _warnWorkSeconds;
int _gateMarkerMask;
int _promoteOfferMask;
float _firstPromoteT = -1f;
int _promoteCount;
EndingChoice _ending;
int _retireCount;

// ----- red line 11 timing capture -----
public float HookReadyT { get; private set; } = -1f;
public float FirstWageT { get; private set; } = -1f;
public float FirstOpT { get; private set; } = -1f;
public bool FirstWageThreeSync { get; private set; }
public int WageBeatCount { get; private set; }

// ----- T007 skill tree + tutorial face -----
bool _tutorial;
bool _tutorialGuideDone;
float _tutorialGuideT = -1f;
public float FirstSkillUpT { get; private set; } = -1f;

```

```

public int SkillUpgradeCount { get; private set; }
public bool TutorialGuideDone => _tutorialGuideDone;
public float TutorialGuideT => _tutorialGuideT;

/// <summary>T007: fires exactly once when the day-one skill guide beat lands.</summary>
public event Action<float> OnTutorialGuide;

// ----- T006-a: SYS04 workplace event flow -----
readonly WorkerEventFlow _flow = new WorkerEventFlow();
readonly List<WorkerEventFlow.ResolvedEvent> _resolvedScratch = new List<WorkerEventFlow.ResolvedEvent>(8);
int _slackSessions;
float _idleSeconds;
float _freezeRemaining;

/// <summary>
/// One settle beat. The presentation must answer THIS SAME beat with all three
/// faces in the same frame: the readout number, the pulse punch and the sound.
/// (The rendered-frame and audible co-hit claims belong to the RT slice; the
/// sim owns the contract, WorkerMainHost consumes it.)
/// </summary>
public struct WageBeat
{
    public float T;
    public double Amount;
    public double CoinAfter;
    public bool Number;
    public bool Pulse;
    public bool Sound;
}

public event Action<WageBeat> OnWageBeat;

/// <summary>T006-a: fires the same tick a workplace event card resolves.</summary>
public event Action<WorkerEventFlow.ResolvedEvent> OnEventCard;

/// <summary>T008-b2: fires when the flip-show tuple changes inside a tick;
/// (striking, warned) is the live show state - the host owns the card face
/// and the recovery toasts (a show, never a lockout).</summary>
public event Action<bool, bool> OnFlipShow;

/// <summary>T008-b2: fires the same tick a promotion lands (new rank index),
/// after the ceremony-card drive so the host trio (red dot + horn + badge
/// swap) lands on the same frame as the promotion itself.</summary>
public event Action<int> OnPromoted;

// ----- T010 slice 2: promotion funnel (doc section 8 evt_promote) -----
static readonly List<string> PromoteFunnelLog = new List<string>(16);

/// <summary>Runtime telemetry sink: the host wires this to Sy.Telemetry
/// (EventIds.Promote with state + game fields); the funnel log itself

```

```

/// stays the deterministic check canary.</summary>
public static event Action<string> OnPromoteFunnel;

/// <summary>Promotion funnel trail since the last clear
/// ("state:g11_promote" rows), consumed by ShareCardCheck.</summary>
public static IReadOnlyList<string> PromoteFunnel => PromoteFunnelLog;

/// <summary>Test probe: wipes the promotion funnel log only.</summary>
public static void ClearPromoteFunnelForTests() => PromoteFunnelLog.Clear();

static void TrackPromote(string state)
{
    PromoteFunnelLog.Add(state + ":g11_promote");
    OnPromoteFunnel?.Invoke(state);
}

// ----- T013: the two remaining IAA points (doc section 6 rows 3-4) -----
/// <summary>Event-card skip point id (funnel + AdCap namespace, g11_-isolated).</summary>
public const string SkipPointId = "g11_event_skip";
/// <summary>30-minute wage boost point id (funnel + AdCap namespace, g11_-isolated).</summary>
public const string BoostPointId = "g11_boost_30m";

float _skipCooldownRemaining;
float _boostRemaining;
int _skipCount;
int _boostCount;

static readonly List<string> IaaFunnelLog = new List<string>(32);

/// <summary>Runtime telemetry sink for both T013 points: the host wires
/// this to Sy.Telemetry (EventIds.Ad with state + point + game fields);
/// the funnel log itself stays the deterministic 08/09 canary.</summary>
public static event Action<string, string> OnIaaFunnel;

/// <summary>Ad funnel trail since the last clear ("state:pointId" rows),
/// consumed by OfflineCheck 08/09.</summary>
public static IReadOnlyList<string> IaaFunnel => IaaFunnelLog;

/// <summary>Test probe: wipes the T013 funnel log only.</summary>
public static void ClearIaaFunnelForTests() => IaaFunnelLog.Clear();

static void TrackIaa(string state, string pointId)
{
    IaaFunnelLog.Add(state + ":" + pointId);
    OnIaaFunnel?.Invoke(state, pointId);
}

/// <summary>Verdict of the ad-gated event-skip resolution.</summary>
public enum SkipResult { Granted = 0, Blocked = 1, AdFail = 2, InWindow = 3 }

```

```

/// <summary>Verdict of the ad-gated wage-boost resolution.</summary>
public enum BoostResult { Granted = 0, Blocked = 1, AdFail = 2, InWindow = 3 }

/// <summary>Skip point live window face: true while the cooldown gate
/// holds (the host dims the button - the dim IS the honest hint).</summary>
public bool SkipInCooldown => _skipCooldownRemaining > 0f;
public float SkipCooldownRemaining => _skipCooldownRemaining;
public int SkipCount => _skipCount;
/// <summary>Boost point live window face: true while the x2 wage
/// acceleration runs.</summary>
public bool BoostActive => _boostRemaining > 0f;
public float BoostRemaining => _boostRemaining;
public int BoostCount => _boostCount;

/// <summary>
/// T013 event-skip resolution (doc section 6 row 3, rewarded_skip 8/day
/// cooldown-gated): pre-offer window gate first (inside the cooldown the
/// offer never opens - zero state, zero funnel, the dimmed button is the
/// law), then the honest category hint, then the ad verdict, then the
/// local cap consume - and only a grant resolves. A grant ends the
/// current event's show at once (an armed freeze window lifts) and arms
/// the skip cooldown; scheduled chains keep their own beats per rules.
/// Every refused path changes zero state.
/// </summary>
public SkipResult TrySkip(AdCapPolicy policy, string dayKey, bool adWatched)
{
    if (_skipCooldownRemaining > 0f) return SkipResult.InWindow;
    TrackIaa("offer", SkipPointId);
    TrackIaa("accept", SkipPointId);
    string category = AdCategories.CategoryOfPoint(SkipPointId);
    if (policy != null && !policy.HasCategoryBudget(dayKey, category))
    {
        TrackIaa("blocked", SkipPointId);
        return SkipResult.Blocked;
    }
    if (!adWatched)
    {
        TrackIaa("ad_fail", SkipPointId);
        return SkipResult.AdFail;
    }
    if (policy != null && !policy.TryConsumeRewarded(dayKey, category))
    {
        TrackIaa("blocked", SkipPointId);
        return SkipResult.Blocked;
    }
    _freezeRemaining = 0f;
    _skipCooldownRemaining = WorkerData.SkipCooldownSec;
    _skipCount++;
    TrackIaa("granted", SkipPointId);
    return SkipResult.Granted;
}

```

```

}

/// <summary>
/// T013 wage-boost resolution (doc section 6 row 4, rewarded_boost 6/day):
/// pre-offer window gate first (a repeat tap while the x2 runs refuses
/// pre-offer with zero state change), then the honest category hint, then
/// the ad verdict, then the local cap consume - and only a grant arms the
/// 30-minute x2 wage window. Every refused path changes zero state.
/// </summary>
public BoostResult TryBoost(AdCapPolicy policy, string dayKey, bool adWatched)
{
    if (_boostRemaining > 0f) return BoostResult.InWindow;
    TrackIaa("offer", BoostPointId);
    TrackIaa("accept", BoostPointId);
    string category = AdCategories.CategoryOfPoint(BoostPointId);
    if (policy != null && !policy.HasCategoryBudget(dayKey, category))
    {
        TrackIaa("blocked", BoostPointId);
        return BoostResult.Blocked;
    }
    if (!adWatched)
    {
        TrackIaa("ad_fail", BoostPointId);
        return BoostResult.AdFail;
    }
    if (policy != null && !policy.TryConsumeRewarded(dayKey, category))
    {
        TrackIaa("blocked", BoostPointId);
        return BoostResult.Blocked;
    }
    _boostRemaining = WorkerData.BoostDurationSec;
    _boostCount++;
    TrackIaa("granted", BoostPointId);
    return BoostResult.Granted;
}

/// <summary>T013: external card injection - an engine-resolved card from
/// outside the tick stream (machine-gate driver, LiveOps seat) travels
/// the SAME fx/freeze/card application path a tick card travels, so
/// injected cards behave exactly like tick cards.</summary>
public void NotifyExternalCard(WorkerEventFlow.ResolvedEvent card)
{
    _resolvedScratch.Clear();
    _resolvedScratch.Add(card);
    ApplyResolved(_resolvedScratch);
}

public double Coin => _coin;
public double Exp => _exp;
public double SimTime => _simTime;

```

```

public double WageBeatCarry => _wageBeatCarry;
public int NgLoop => _ngLoop;
public float Kpi => _kpi;
public float Mood => _mood;
public float Insp => _insp;
public bool Working => _working;
public int RankIndex => _rankIndex;
public int SlackSessions => _slackSessions;
public float IdleSeconds => _idleSeconds;
public float FreezeRemaining => _freezeRemaining;
public float FirstEventCardT => _flow.FirstCardT;
public IReadOnlyList<WorkerEventFlow.LogEntry> EventLog => _flow.Log;

// ----- T008 promotion / flip-show face (slice a: sim law, host lands with slice b) -----
public bool IsStriking => _strike;
public int StrikeCount => _strikeCount;
public bool IsWarned => _warn;
public int WarnCount => _warnCount;
public float WarnWorkSeconds => _warnWorkSeconds;
public int GateMarkerMask => _gateMarkerMask;
public float FirstPromoteT => _firstPromoteT;
public int PromoteCount => _promoteCount;
public EndingChoice CurrentEnding => _ending;
public int RetireCount => _retireCount;
public bool EndingChoiceOpen
    => _rankIndex >= (int)WorkerData.WorkerRank.Ceo && _ending == EndingChoice.None;

public WorkerSim() : this(false) { }

/// <summary>Tutorial mode is opt-in so the frozen gate scenarios (Redline11Check
/// main run, EventEngineCheck) keep their byte-exact baseline; the live host and
/// the SkillCheck first-skill scenario opt in.</summary>
public WorkerSim(bool withTutorial)
{
    _tutorial = withTutorial;
    _kpi = WorkerData.InitKpi;
    _mood = WorkerData.InitMood;
    _insp = WorkerData.InitInsp;
    HookReadyT = 0f; // HUD content is live from frame zero: no splash, no wall
}

/// <summary>First core op (Work/Slack toggle). No unlock, no cost, usable from t=0.
/// Op semantics owned here: one counted slack session per Work->Slack flip
/// (evt_04) and any op resets the idle timer (evt_05).</summary>
public void SetWorking(bool working)
{
    if (working != _working && !working) _slackSessions++;
    _working = working;
    _idleSeconds = 0f;
    if (FirstOpT < 0f) FirstOpT = (float)_simTime;
}

```

```

}

// ----- T007: comedy skill tree face (doc section 3, 4 branches x 3 levels) -----

/// <summary>Learned levels of a branch (0 = not learned).</summary>
public int SkillLevel(WorkerData.SkillBranch branch) => _skillLevels[(int)branch];

/// <summary>Next-level cost of a branch; -1 when the branch is maxed.</summary>
public int SkillUpgradeCost(WorkerData.SkillBranch branch)
{
    int lv = _skillLevels[(int)branch];
    return lv >= WorkerData.SkillLevelCount ? -1 : WorkerData.SkillUpgradeExpCost[lv];
}

/// <summary>Event-face exp reward channel (doc section 1: skill currency =
/// idle accrual + event rewards). Deterministic driver for tutorial and later
/// system slices (week/annual gifts land with T008).</summary>
public void GrantExp(double amount)
{
    if (amount <= 0.0) return;
    _exp = Math.Min(_exp + amount, WorkerData.ExpCap);
}

/// <summary>T009 offline-gain deposit channel: the offline claim banks
/// through the real wallet (never a parallel ledger), clamped at the coin
/// cap; non-positive amounts keep zero state change.</summary>
public void DepositCoin(double amount)
{
    if (amount <= 0.0) return;
    _coin = Math.Min(_coin + amount, WorkerData.CoinCap);
}

/// <summary>Sequential upgrade attempt: L0->L1->L2->L3 only (no skipping),
/// exact exp cost, refused with zero state change when broke or maxed.
/// A successful upgrade stamps the red-line first-skill beat, counts it and
/// drives the SYS04 skill-up hook (evt_11 slot, EventEngineCheck E11 law).</summary>
public SkillUpResult TryUpgradeSkill(WorkerData.SkillBranch branch)
{
    int idx = (int)branch;
    int lv = _skillLevels[idx];
    if (lv >= WorkerData.SkillLevelCount) return SkillUpResult.Maxed;
    int cost = WorkerData.SkillUpgradeExpCost[lv];
    if (_exp < cost) return SkillUpResult.NoExp;
    _exp -= cost;
    _skillLevels[idx] = lv + 1;
    if (FirstSkillUpT < 0f) FirstSkillUpT = (float)_simTime;
    SkillUpgradeCount++;
    NotifySkillUpgraded();
    return SkillUpResult.Ok;
}

```

```

/// <summary>Idempotent delta settle: continuous accrual, order-independent inputs.</summary>
public void Tick(float dt)
{
    if (dt <= 0f) return;

    // T013: both IAA windows burn on sim time in every branch - they are
    // real-time offers, so they keep counting down inside the freeze show
    bool boosting = _boostRemaining > 0f;
    if (boosting) _boostRemaining = Math.Max(0f, _boostRemaining - dt);
    if (_skipCooldownRemaining > 0f)
        _skipCooldownRemaining = Math.Max(0f, _skipCooldownRemaining - dt);

    // T006-a forced-rest show (evt_07): progress frozen, time keeps flowing
    if (_freezeRemaining > 0f)
    {
        _freezeRemaining -= dt;
        _simTime += dt;
        _idleSeconds += dt;
        return;
    }

    // wage main axis: base x rank coef x Ppt skill mult x NG+ inherit (four-factor chain)
    // T013: the boost window rides the whole chain as a flat multiplier;
    // with no boost the x1 multiply is byte-identical to the raw rate
    float rate = WorkerData.WagePerSec(_rankIndex, _skillLevels[(int)WorkerData.SkillBranch.Ppt], _ngLoop)
        * (boosting ? WorkerData.BoostWageMult : 1f);
    _coin = Math.Min(_coin + (double)rate * dt, WorkerData.CoinCap);
    _exp = Math.Min(_exp + WorkerData.ExpPerSecOnline * dt, WorkerData.ExpCap);

    // dual-axis deltas (doc section 2) + T008 flip shows: during the strike show
    // the mood axis suspends its mode delta and only the natural +0.5/s regen
    // applies, so every strike always lifts at 25 within ~10s of show time -
    // the red law is "a show, never a lockout". The warn show keeps every rate
    // untouched; only its own recovery rule (10s continuous Work) runs.
    int report = _skillLevels[(int)WorkerData.SkillBranch.Report];
    int slackArt = _skillLevels[(int)WorkerData.SkillBranch.SlackArt];
    // T008-b2: the host hears the flip-show tuple only when it truly flips
    bool showBefore = _strike;
    bool warnBefore = _warn;
    if (!_strike && _mood < WorkerData.MoodStrikeLine) { _strike = true; _strikeCount++; }
    if (!_warn && _kpi < WorkerData.KpiWarnLine) { _warn = true; _warnCount++; _warnWorkSeconds = 0f; }
    float moodDelta = _strike ? WorkerData.MoodRecoverPerSec : WorkerData.MoodPerSec(_working, slackArt);
    _kpi = ClampAxis(_kpi + WorkerData.KpiPerSec(_working, report, slackArt) * dt);
    _mood = ClampAxis(_mood + moodDelta * dt);
    _insp = ClampAxis(_insp + WorkerData.InspPerSec(_working) * dt);
    if (_strike && _mood >= WorkerData.MoodRecoverTo) _strike = false;
    if (_warn)
    {
        if (_working)

```

```

        {
            _warnWorkSeconds += dt;
            if (_warnWorkSeconds >= WorkerData.KpiRecoverWorkSeconds) _warn = false;
        }
        else _warnWorkSeconds = 0f;
    }
    if (_strike != showBefore || _warn != warnBefore) OnFlipShow?.Invoke(_strike, _warn);
    TryPromote();

    _simTime += dt;

    // T007 tutorial beat: the day-one red packet lands once inside the T+3-10s
    // window and funds exactly one L1 skill (event-face exp reward, doc section 3:
    // skill currency = idle accrual + event rewards).
    if (_tutorial && !_tutorialGuideDone && _simTime >= TutorialGuideAtSec)
    {
        _tutorialGuideDone = true;
        _tutorialGuideT = (float)_simTime;
        _exp = Math.Min(_exp + TutorialRedPacketExp, WorkerData.ExpCap);
        OnTutorialGuide?.Invoke(_tutorialGuideT);
    }

    _wageBeatCarry += dt;
    while (_wageBeatCarry >= WageBeatPeriodSec)
    {
        _wageBeatCarry -= WageBeatPeriodSec;
        EmitBeat(rate);
    }

    _idleSeconds += dt;
    TickFlow(dt);
}

/// <summary>T006-a: drive the SYS04 flow and answer every resolved card the
/// same tick - fx land on the axes (clamped), freeze windows arm, the host
/// hears the card through OnEventCard.</summary>
void TickFlow(float dt)
{
    _flow.Tick(BuildFlowState(dt), _resolvedScratch);
    ApplyResolved(_resolvedScratch);
}

/// <summary>T006-b driver slice: the later system slices (week rollover,
/// skill upgrade, annual settlement, promotion) translate their system beats
/// into flow hook calls through these notify methods; the sim owns the
/// fx/freeze/card application path so hook cards behave exactly like tick
/// cards. T007 wires skill upgrades, T008 wires week/year/promotion.</summary>
public void NotifyWeekPassed() => DriveHook(_flow.OnWeekPassed);
public void NotifySkillUpgraded() => DriveHook(_flow.OnSkillUpgraded);
public void NotifyAnnualSettle() => DriveHook(_flow.OnAnnualSettle);

```

```

public void NotifyPromotion(int newRankIndex)
{
    _rankIndex = newRankIndex;
    DriveHook(_flow.OnStageGateMet);
}

// ----- T008-a: promotion gates (doc section 4, dual track) + gate-4 dual ending -----

/// <summary>Gate-4 dual ending (doc section 4): the choice stays open at the CEO
/// rank. CeoRun keeps the endless run; Retire plays the financial-freedom ending
/// and rolls a NG+ loop: rank/axes/skills/exp reset, event flags cleared, wage
/// inherits x(1+0.10 per loop); coin stays - the career resets, the wallet does
/// not (share-card history keeps with T010). The ending slot itself resets to
/// None with the career, so the next run's CEO climb re-opens the choice
/// (multi-loop law: RetireCount/NG+ wage carry the history, not the slot).</summary>
public enum EndingChoice { None = 0, CeoRun = 1, Retire = 2 }

/// <summary>Resolves the open ending; refused with zero state change when the
/// choice is not open (not at CEO yet, or already decided).</summary>
public bool ChooseEnding(EndingChoice choice)
{
    if (_ending != EndingChoice.None) return false;
    if (_rankIndex < (int)WorkerData.WorkerRank.Ceo || choice == EndingChoice.None) return false;
    _ending = choice;
    if (choice == EndingChoice.CeoRun) return true;
    _retireCount++;
    _ngLoop++;
    _rankIndex = 0;
    _kpi = WorkerData.InitKpi;
    _mood = WorkerData.InitMood;
    _insp = WorkerData.InitInsp;
    _exp = 0.0;
    Array.Clear(_skillLevels, 0, _skillLevels.Length);
    _gateMarkerMask = 0;
    _promoteOfferMask = 0;
    _strike = false;
    _warn = false;
    _warnWorkSeconds = 0f;
    _flow.Reset();
    _ending = EndingChoice.None;
    return true;
}

/// <summary>Promotion law (dual track): gates 0/1/3 promote only after their
/// marker card resolved (evt_04 slack rhythm / evt_02b weekly chain / evt_09
/// annual settle) AND the KPI threshold is met; gate 2's marker evt_12 is
/// stage-locked to the director rank by the frozen event table, so it IS the
/// ceremony card that fires through NotifyPromotion's OnStageGateMet drive at
/// the new rank - the KPI edge alone carries that gate. Whichever track lands
/// second fires the ceremony the same tick (promotion pacing: gate 1 inside P1).</summary>

```

```

void TryPromote()
{
    int gate = _rankIndex;
    if (gate >= WorkerData.GateKpiThreshold.Length) return;
    bool markerOk = gate == 2 || (_gateMarkerMask & (1 << gate)) != 0;
    bool kpiOk = _kpi >= WorkerData.GateKpiThreshold[gate];
    // T010 slice 2: the funnel exposure row - the first moment either
    // promotion track lands at this gate (marker card armed or KPI over
    // the line) is the "offer" face of the promotion journey.
    if ((_promoteOfferMask & (1 << gate)) == 0 && (markerOk || kpiOk))
    {
        _promoteOfferMask |= 1 << gate;
        TrackPromote("offer");
    }
    if (!markerOk || !kpiOk) return;
    TrackPromote("ready");
    if (_firstPromoteT < 0f) _firstPromoteT = (float)_simTime;
    _promoteCount++;
    NotifyPromotion(gate + 1);
    TrackPromote("promoted");
    OnPromoted?.Invoke(_rankIndex);
}

/// <summary>Gate marker bookkeeping: each frozen marker id (evt_04/evt_02b/
/// evt_12/evt_09) arms its gate bit the moment its card resolves.</summary>
void TrackGateMarker(string id)
{
    int gate = Array.IndexOf(WorkerData.GateMarkerEventId, id);
    if (gate >= 0) _gateMarkerMask |= 1 << gate;
}

void DriveHook(Action<WorkerEventFlow.FlowState, List<WorkerEventFlow.ResolvedEvent>> hook)
{
    _resolvedScratch.Clear();
    hook(BuildFlowState(0f), _resolvedScratch);
    ApplyResolved(_resolvedScratch);
}

WorkerEventFlow.FlowState BuildFlowState(float dt)
{
    return new WorkerEventFlow.FlowState
    {
        Dt = dt,
        SimTime = (float)_simTime,
        Kpi = _kpi,
        Mood = _mood,
        Insp = _insp,
        RankIndex = _rankIndex,
        SlackSessions = _slackSessions,
        IdleSeconds = _idleSeconds
    }
}

```

```

    };
}

void ApplyResolved(List<WorkerEventFlow.ResolvedEvent> resolved)
{
    for (int i = 0; i < resolved.Count; i++)
    {
        var e = resolved[i];
        _kpi = ClampAxis(_kpi + e.Kpi);
        _mood = ClampAxis(_mood + e.Mood);
        _insp = ClampAxis(_insp + e.Insp);
        if (e.FreezeSeconds > 0f) _freezeRemaining = e.FreezeSeconds;
        TrackGateMarker(e.Id);
        OnEventCard?.Invoke(e);
    }
}

void EmitBeat(float rate)
{
    WageBeatCount++;
    var beat = new WageBeat
    {
        T = (float)_simTime,
        Amount = (double)rate * WageBeatPeriodSec,
        CoinAfter = _coin,
        Number = true,
        Pulse = true,
        Sound = true
    };
    if (FirstWageT < 0f)
    {
        FirstWageT = beat.T;
        FirstWageThreeSync = beat.Number && beat.Pulse && beat.Sound && beat.Amount > 0;
    }
    OnWageBeat?.Invoke(beat);
}

// ----- T012: save mirror restore face (red line 12: local authority) -----

/// <summary>Restore the frozen save mirror into this sim, field-exact on
/// the whole doc-section-10 face plus the two determinism carries, so a
/// resumed run continues the same beat/op stream. Boundary: the event
/// slate restarts fresh (flow internals are outside the mirror), the
/// transient show/ceremony faces re-derive on their own - flip-shows
/// re-arm from the axes on the next tick, gate markers re-arm through
/// new event cards, the funnel and first-beat stamps restart with the
/// resumed window. A null save (corrupt fallback) is a fresh-start
/// no-op restore.</summary>
public void Restore(WorkerSaveData save)
{

```

```

        if (save == null) return;
        _coin = save.coin;
        _exp = save.exp;
        _simTime = save.simTime;
        _wageBeatCarry = save.wageBeatCarry;
        _kpi = save.kpi;
        _mood = save.mood;
        _insp = save.insp;
        _working = save.working;
        _rankIndex = save.rankIndex;
        _ngLoop = save.ngLoop;
        _retireCount = save.retireCount;
        if (save.skillLevels != null)
            for (int i = 0; i < _skillLevels.Length && i < save.skillLevels.Length; i++)
                _skillLevels[i] = save.skillLevels[i];
        _strike = false;
        _strikeCount = 0;
        _warn = false;
        _warnCount = 0;
        _warnWorkSeconds = 0f;
        _gateMarkerMask = 0;
        _promoteOfferMask = 0;
        _firstPromoteT = -1f;
        _promoteCount = 0;
        _ending = EndingChoice.None;
        _freezeRemaining = 0f;
        _idleSeconds = 0f;
        _slackSessions = 0;
        // T013: both IAA windows are transient session faces - they restart
        // empty on resume exactly like the show/ceremony faces above
        _skipCooldownRemaining = 0f;
        _boostRemaining = 0f;
        _flow.Reset();
    }

    /// <summary>Invariant round-trip fingerprint over the full live state: the dual-run byte-parity face.</summary>
    public string StateFingerprint()
    {
        var sb = new StringBuilder();
        sb.Append("v").Append(WorkerData.SchemaVersion).Append(';');
        sb.Append("t:").Append(_simTime.ToString("R", Inv)).Append(';');
        sb.Append("coin:").Append(_coin.ToString("R", Inv)).Append(';');
        sb.Append("exp:").Append(_exp.ToString("R", Inv)).Append(';');
        sb.Append("carry:").Append(_wageBeatCarry.ToString("R", Inv)).Append(';');
        sb.Append("axis:").Append(_kpi.ToString("R", Inv)).Append(',')
            .Append(_mood.ToString("R", Inv)).Append(',').Append(_insp.ToString("R", Inv)).Append(';');
        sb.Append("mode:").Append(_working ? "work" : "slack").Append(';');
        sb.Append("rank:").Append(_rankIndex).Append(",ng:").Append(_ngLoop).Append(';');
        sb.Append("skills:").Append(_skillLevels[0]).Append(',').Append(_skillLevels[1])
            .Append(',').Append(_skillLevels[2]).Append(',').Append(_skillLevels[3]).Append(';');
    }

```

```

        sb.Append("r11:").Append(HookReadyT.ToString("R", Inv)).Append(',')
        .Append(FirstWageT.ToString("R", Inv)).Append(',').Append(FirstOptT.ToString("R", Inv))
        .Append(',').Append(WageBeatCount).Append(';');
        sb.Append("ops:").Append(_slackSessions).Append(',').Append(_idleSeconds.ToString("R", Inv)).Append(';');
        sb.Append("frz:").Append(_freezeRemaining.ToString("R", Inv)).Append(';');
        sb.Append("t8:").Append(_strike ? 1 : 0).Append(',').Append(_strikeCount).Append(',')
        .Append(_warn ? 1 : 0).Append(',').Append(_warnCount).Append(',')
        .Append(_warnWorkSeconds.ToString("R", Inv)).Append(',')
        .Append(_gateMarkerMask).Append(',')
        .Append(_firstPromoteT.ToString("R", Inv)).Append(',').Append(_promoteCount).Append(',')
        .Append((int)_ending).Append(',').Append(_retireCount).Append(';');
        sb.Append("tut:").Append(_tutorialGuideDone ? 1 : 0).Append(',')
        .Append(_tutorialGuideT.ToString("R", Inv)).Append(',')
        .Append(FirstSkillUpT.ToString("R", Inv)).Append(',').Append(SkillUpgradeCount).Append(';');
        sb.Append("iaa:").Append(_skipCooldownRemaining.ToString("R", Inv)).Append(',')
        .Append(_skipCount).Append(',')
        .Append(_boostRemaining.ToString("R", Inv)).Append(',').Append(_boostCount).Append(';');
        sb.Append("evt:").Append(_flow.EventSig());
        return sb.ToString();
    }

    public byte[] StateHash()
    {
        using (var md5 = MD5.Create())
            return md5.ComputeHash(Encoding.UTF8.GetBytes(StateFingerprint()));
    }

    static float ClampAxis(float v)
        => Math.Min(Math.Max(v, WorkerData.AxisMin), WorkerData.AxisMax);

    static readonly CultureInfo Inv = CultureInfo.InvariantCulture;
}

// ===== FILE: Assets_Game\Scripts\Runtime\Game\WorkerTexts.cs =====
using System;
using System.Collections.Generic;
using System.Text;

namespace Sy.G11.Game
{
    /// <summary>
    /// English-source text table (rule 8: English strings live in code, Chinese lives
    /// in the Configs/Texts_zhCN.json data file). Miss degrades to the key itself,
    /// matching the LocaleService contract. LookupWithZh is the ready-made delegate
    /// for LocaleService wiring: zh langs resolve from the loaded data table, other
    /// langs fall back to the English source.
    /// </summary>
    public static class WorkerTexts
    {
        public const string LangZh = "zh-CN";
    }
}

```

```

static readonly (string key, string text)[] EnRows =
{
    ("hook_boss_line", "Day one. Work hard, and I might let you breathe."),
    ("hook_slogan", "Idle office life: paychecks roll, ranks fly, jokes escalate."),
    // T019 polish: strongest-picture flash-forward copy (06-script rule 1,
    // the recording money shot): year-10 CEO penthouse tease + the snap-back
    // line that sells the grind back to day one.
    ("flash_ceo_line", "Flash-forward, year 10: your penthouse. Your name on the door."),
    ("flash_ceo_tag", "Snap back to day one. Grind on, intern."),
    ("rank_intern", "Intern"),
    ("rank_corporate", "Corporate Drone"),
    ("rank_teamlead", "Team Lead"),
    ("rank_director", "Director"),
    ("rank_ceo", "CEO"),
    ("hud_coin", "Wage"),
    ("hud_kpi", "KPI"),
    ("hud_mood", "Mood"),
    ("hud_insp", "Inspiration"),
    ("action_work", "Work"),
    ("action_slack", "Slack"),
    ("skill_ppt", "PPT Mastery"),
    ("skill_report", "The Art of Reporting"),
    ("skill_slackart", "Slackology"),
    ("skill_manageup", "Managing Up"),
    ("skill_ppt_11", "Can Make Slides"),
    ("skill_ppt_12", "Slides Have a Soul"),
    ("skill_ppt_13", "One Slide Beats Ten"),
    ("skill_report_11", "Explains Things"),
    ("skill_report_12", "Boss Loves It"),
    ("skill_report_13", "Reporting Is Art"),
    ("skill_slackart_11", "Looks Busy"),
    ("skill_slackart_12", "Leaves No Trace"),
    ("skill_slackart_13", "Certified Slack Master"),
    ("skill_manageup_11", "Knows How to Complain"),
    ("skill_manageup_12", "Resources Lean Your Way"),
    ("skill_manageup_13", "Boss's Inner Circle"),
    ("warn_mood_strike", "Mood bottomed out: your soul is on strike. Even the succulent is worried."),
    ("warn_mood_recover", "The soul returns to the desk. Mood recovering."),
    ("warn_kpi_letter", "KPI critical! HR is strolling over with a red card that is definitely not a bonus."),
    ("warn_kpi_recover", "Ten seconds of honest work later, the red card is quietly withdrawn."),
    ("promote_toast", "Promoted! New badge, new office, same absurdity."),
    ("ending_ceo", "CEO: the endless corner-office run."),
    ("ending_retire", "Financial freedom: retired with the office succulent."),
    ("ending_choice_title", "Top-Floor Crossroads: Run On or Retire?"),
    ("ending_ceo_toast", "The corner office is yours. The endless run continues."),
    ("ending_retire_toast", "Financial freedom! The wallet keeps its coins, the career starts over."),
    ("offline_summary_header", "While you were slacking, the paycheck kept coming:"),
    ("offline_summary_line", "Away {0} min, paid {1} coins, missed {2} absurd events."),
    ("offline_collect_btn", "Collect Paycheck"),
}

```

```

("offline_double_btn", "Watch Ad: Paycheck x2"),
("offline_collected_toast", "Payday: +{0} coins banked. The succulent approves."),
("offline_doubled_toast", "Overtime magic: +{0} coins. The ad gods smile."),
("offline_ad_fail_toast", "Ad not ready yet. The double offer stays - try again."),
("offline_capped_toast", "Today's double budget is spent. The plain payday still pays."),
("evt_01_name", "Pie-in-the-Sky Meeting"),
("evt_01_line", "Boss unveils a glowing pie chart: IPO next year. Trust the pie."),
("evt_01b_name", "The Pie Came True"),
("evt_01b_line", "Against all odds, the pie chart delivered. KPI rain!"),
("evt_02_name", "Weekly Report Literature Awards"),
("evt_02_line", "Your weekly report wins Best Fiction. It was 40 percent vibes."),
("evt_02b_name", "Praised by the Boss"),
("evt_02b_line", "The boss praises your report. The AI wrote it. Nobody asks."),
("evt_03_name", "Team-Building Social Death"),
("evt_03_line", "Karaoke. In front of everyone. Your manager picked a power ballad."),
("evt_04_name", "Paid Bathroom Break"),
("evt_04_line", "Five slack sessions earned you a throne of tranquility. Motivation restored."),
("evt_05_name", "The Succulent Wilted"),
("evt_05_line", "Your desk succulent has given up, exactly like your Monday self."),
("evt_05b_name", "The Succulent Lives"),
("evt_05b_line", "One drop of water and it is back. Some things are stronger than KPIs."),
("evt_06_name", "Boss Walks By"),
("evt_06_line", "Everyone switches screens at once. A perfectly synchronized ballet."),
("evt_07_name", "Overwork Hospital"),
("evt_07_line", "The drip is labeled deadline. The nurse says rest 30 seconds. Listen to the nurse."),
("evt_08_name", "The Optimization Letter"),
("evt_08_line", "HR sweetly hands you an envelope. Inside: a desk-relocation notice. Classic."),
("evt_09_name", "Year-End Riddle"),
("evt_09_line", "The bonus envelope holds a riddle. It pays {0} times your daily wage."),
("evt_10_name", "Pantry Gossip"),
("evt_10_line", "Tea-room intel unlocks inspiration. Someone microwaved fish. Again."),
("evt_11_name", "No Meeting Room"),
("evt_11_line", "Every room booked. The meeting is now a hallway stand-up. Very agile."),
("evt_12_name", "PUA Counter-Attack"),
("evt_12_line", "You reply: noted, and also no. The director blinks first. Legend status."),
("ad_double_offer", "Double this paycheck? One ad, zero dignity lost."),
("ad_gacha_pull", "Ten-pull the costume closet. Fashion is a workplace weapon."),
("skill_panel_title", "Skill Tree"),
("skill_panel_btn", "Skills"),
("skill_cost_fmt", "Next level: {0} exp"),
("skill_maxed_line", "MAX. The craft is complete."),
("skill_noexp_line", "Need {0} more exp. The grind continues."),
("skill_effect_wage", "Wage x{0}"),
("skill_effect_kpi", "KPI gain x{0}"),
("skill_effect_mood", "Slack mood x{0}"),
("skill_effect_mood_l3", "Slack mood x{0}, KPI loss halved"),
("skill_effect_offline", "Offline gain x{0}"),
("skill_up_toast", "{0} unlocked: {1}!"),
("tutorial_toast_title", "Day-One Red Packet"),
("tutorial_guide_line", "The boss stuffs a red packet in your hand: +{0} exp. Learn your first skill now."),

```

```

("gacha_panel_btn", "Outfits"),
("gacha_panel_title", "Costume Closet"),
("gacha_pull_btn", "Watch Ad: Ten-Pull"),
("gacha_empty_hint", "The closet is empty. One ad = ten pulls, one Epic guaranteed."),
("gacha_pity_line", "House rule: every ten-pull hides at least one Epic. Jackpot is a labor law."),
("gacha_stats_fmt", "Lifetime pulls {0} 路 Epics {1} 路 {2} pulls since last Epic"),
("offline_share_btn", "Brag Card"),
("offline_share_toast", "Brag card ready - go make the group chat jealous."),
("offline_share_fail_toast", "The card printer jammed. Nothing lost - try again."),
("gacha_granted_toast", "Ten fresh looks drop. The closet approves."),
("gacha_ad_fail_toast", "Ad not ready yet. The ten-pull stays - try again."),
("gacha_capped_toast", "Today's ten-pull budget is spent. Fashion resumes tomorrow."),
("skip_card_btn", "Skip This Meeting (Ad)"),
("skip_granted_toast", "Meeting skipped. Back to pretending to work."),
("skip_ad_fail_toast", "Ad not ready. The meeting drags on - try again."),
("skip_capped_toast", "Today's skip budget is spent. The show must go on."),
("skip_window_toast", "Skip is recharging. Even escape has a cooldown."),
("boost_btn", "2x Wage 30min (Ad)"),
("boost_active_fmt", "2x Wage ON - {0} min left"),
("boost_granted_toast", "Overtime approved: wages x2 for 30 minutes."),
("boost_ad_fail_toast", "Ad not ready. The raise waits - try again."),
("boost_capped_toast", "Today's boost budget is spent. Grind resumes tomorrow."),
("boost_window_toast", "Boost already running. Let it cook."),
("share_offline_title", "Slacked {0} min. The office paid anyway."),
("share_offline_line_coins", "+{0} coins banked while nobody watched"),
("share_offline_line_events", "{0} absurd meetings happened without you"),
("share_offline_line_hook", "Clock in. Slack off. Get promoted."),
("skin_wrinkled_shirt", "The Wrinkled Shirt"),
("skin_coffee_stain_tie", "The Coffee-Stain Tie"),
("skin_free_conf_shirt", "Free Conference Tee"),
("badge_plastic_lanyard", "Plastic Lanyard"),
("badge_carpool_sticker", "Carpool Club Sticker"),
("skin_short_sleeve_suit", "Short-Sleeve Suit"),
("skin_gym_bro_fit", "Gym-Bro Fit"),
("skin_silent_spec_auth", "Silent Spec Author"),
("badge_silver_lanyard", "Silver Lanyard"),
("skin_ceo_golden_suit", "CEO Golden Suit"),
("badge_penthouse_keycard", "Penthouse Keycard"),
("quest_panel_btn", "Quests"),
("quest_panel_title", "Daily Grind & Badge Wall"),
("quest_kind_slacks", "Touch the fish: {0}/{1} slack flips"),
("quest_kind_events", "Survive the meetings: {0}/{1} event cards"),
("quest_kind_skills", "Learn on the job: {0}/{1} skill-ups"),
("quest_kind_wages", "Collect paychecks: {0}/{1} wage beats"),
("quest_kind_promos", "Climb the ladder: {0}/{1} promotion"),
("quest_reward_fmt", "Reward: +{0} coins, +{1} exp"),
("quest_claimed_line", "DONE - the badge wall nods."),
("quest_claim_toast", "Quest cashed: +{0} coins. The grind approves."),
("quest_progress_toast", "Still grinding: {0}/{1}."),
("badge_section_title", "Badge Wall"),

```

```

        ("badge_unlock_toast", "Badge unlocked: {0}!"),
        ("badge_locked_desc", "Keep grinding to earn this one."),
        ("badge_first_wage_name", "First Paycheck"),
        ("badge_first_wage_desc", "The first coin always hits hardest."),
        ("badge_pie_survivor_name", "Pie Meeting Survivor"),
        ("badge_pie_survivor_desc", "Sat through the glowing pie chart. Still kind of believes it."),
        ("badge_first_skill_name", "Crafts a Skill"),
        ("badge_first_skill_desc", "Learned something on company time. HR calls it growth."),
        ("badge_ladder_climber_name", "Ladder Climber"),
        ("badge_ladder_climber_desc", "One rung up. Same desk energy, better badge."),
        ("badge_toilet_master_name", "Paid Toilet Master"),
        ("badge_toilet_master_desc", "25 paid bathroom breaks. The throne room is yours."),
        ("badge_slack_grandmaster_name", "Slack Grandmaster"),
        ("badge_slack_grandmaster_desc", "100 slack flips and nobody noticed a thing."),
        ("badge_meeting_marathoner_name", "Meeting Marathoner"),
        ("badge_meeting_marathoner_desc", "50 absurd meetings survived. This could have been an email."),
        ("badge_paycheck_collector_name", "Paycheck Collector"),
        ("badge_paycheck_collector_desc", "600 wage beats banked. The succulent is proud."),
        ("badge_epic_wardrobe_name", "Epic Wardrobe"),
        ("badge_epic_wardrobe_desc", "Pulled an Epic fit. The office catwalk awaits."),
        ("badge_financial_freedom_name", "Financial Freedom"),
        ("badge_financial_freedom_desc", "Retired once. The wallet remembers.")
    };

    static readonly Dictionary<string, string> En = BuildDict(EnRows);
    static Dictionary<string, string> Zh;

    static Dictionary<string, string> BuildDict((string key, string text)[] rows)
    {
        var d = new Dictionary<string, string>(StringComparer.Ordinal);
        for (int i = 0; i < rows.Length; i++) d[rows[i].key] = rows[i].text;
        return d;
    }

    /// <summary>Total English rows (dup keys collapse in the dict, so check compares both).</summary>
    public static int EnRowCount => EnRows.Length;

    /// <summary>Unique English keys after dict collapse.</summary>
    public static int EnUniqueKeyCount => En.Count;

    public static System.Collections.Generic.IReadOnlyList<string> EnKeys
    {
        get
        {
            var keys = new List<string>(EnRows.Length);
            for (int i = 0; i < EnRows.Length; i++) keys.Add(EnRows[i].key);
            return keys;
        }
    }
}

```

```

/// <summary>English source read; miss degrades to the key (LocaleService contract).</summary>
public static string GetEn(string key) => En.TryGetValue(key, out var t) && !string.IsNullOrEmpty(t) ? t : key;

/// <summary>Parse the flat zhCN data table (Configs/Texts_zhCN.json) into the lookup.</summary>
public static void LoadZhTable(string flatJson)
{
    var parsed = ParseFlatJson(flatJson);
    lock (Gate)
    {
        Zh = parsed;
    }
}

/// <summary>Count of loaded zh keys (0 when the data table was never loaded).</summary>
public static int ZhKeyCount { get { lock (Gate) { return Zh != null ? Zh.Count : 0; } } }

/// <summary>LocaleService lookup delegate: zh langs read the data table, others read English.</summary>
public static string LookupWithZh(string lang, string key)
{
    bool zh = !string.IsNullOrEmpty(lang) && lang.IndexOf("zh", StringComparison.OrdinalIgnoreCase) >= 0;
    if (zh)
    {
        lock (Gate)
        {
            if (Zh != null && Zh.TryGetValue(key, out var t) && !string.IsNullOrEmpty(t)) return t;
        }
    }
    return GetEn(key);
}

static readonly object Gate = new object();

/// <summary>Minimal flat-JSON parser: {"k":"v","k2":"v2"} with basic escapes, no nesting.</summary>
static Dictionary<string, string> ParseFlatJson(string json)
{
    var result = new Dictionary<string, string>(StringComparer.Ordinal);
    if (string.IsNullOrEmpty(json)) return result;
    int i = 0;
    int n = json.Length;
    while (i < n)
    {
        int keyStart = json.IndexOf('\"', i);
        if (keyStart < 0) break;
        int keyEnd = json.IndexOf('\"', keyStart + 1);
        if (keyEnd < 0) break;
        int colon = json.IndexOf(':', keyEnd + 1);
        if (colon < 0) break;
        int valStart = json.IndexOf('\"', colon + 1);
        if (valStart < 0) break;
        int valEnd = valStart + 1;
    }
}

```

```

var val = new StringBuilder();
while (valEnd < n)
{
    char c = json[valEnd];
    if (c == '\\')
    {
        char next = valEnd + 1 < n ? json[valEnd + 1] : '\0';
        if (next == 'n') val.Append('\n');
        else if (next == 't') val.Append('\t');
        else val.Append(next);
        valEnd += 2;
        continue;
    }
    if (c == '"') break;
    val.Append(c);
    valEnd++;
}
result[json.Substring(keyStart + 1, keyEnd - keyStart - 1)] = val.ToString();
i = valEnd + 1;
}
return result;
}
}
}

```